

Vorlesung 5: Qualitätssicherung

Prof. Dr. Falk Howar

Automated Quality Assurance
Lehrstuhl für Software Engineering
Fakultät für Informatik
TU Dortmund



Logik-basierte Spezifikation.

- Formale Semantik logischer Ausdrücke **benennen** und **erläutern**



- Einfache Eigenschaften des Ein-/Ausgabe-Verhaltens einer Komponente in formaler Logik **ausdrücken**



- Formale Methoden zur Bewertung von Eigenschaften logischer Ausdrücke **benennen** und **anwenden**



Modellierung / Meta-Modellierung.

- Verhalten einfacher Komponenten in Modellen **spezifizieren**



- (Formale) Methoden zur Ausführung und Komposition von Modellen einfacher Komponenten **erläutern** und **anwenden**



Analyse von Modellen.

- Bounded Model Checking und Invariantenbeweise zur Überprüfung von Eigenschaften von Komponenten **anwenden**



Lernziele

- Repräsentationen eines Programmes, auf denen Verifikation stattfinden kann, **benennen** und **verstehen**



- Metriken auf Basis der CFG **ausrechnen**



- Spezifikationen in strukturierter operationeller Semantik **verstehen** und **erläutern**



- Beweise für mögliche Ausführung von While Programmen mit Hilfe von S.O.S. Regeln **durchführen**



- Symbolische Ausführung für While Programme **durchführen**



Agenda

1. Motivation und Begriffe
2. (Code) Standards
3. (Code) Reviews
4. Modellierung und Generierung
5. Verifikation: Model-Checking und Invarianten
6. Metriken und Statische Code Analyse
7. Formale Semantik and Symbolische Ausführung
8. Literatur und Standards

Motivation

Software ist omnipräsent in modernen Geräten!

Bugs auch ...

Bugs im Betrieb können wirklich teuer werden!

⇒ Methoden, um Fehler zu finden und zu verhindern

Ariane 5

- Junferflug 4. Juni, 1996
- Explodiert 40 Sekunden nach Start
- Lange teuerster Software Fehler : 500M USD



Fehlerhafte Integration von (unnötigem) Legacy Code:

- Unbehandelte Exception in floating-point Umwandlung
- Umwandlung von a 64-bit int to a 16-bit signed integer
- Funktioniert für Zahlen kleiner oder gleich 2^{15}
- Größere Zahl $\Rightarrow$ Exception ...

Heartbleed

- Schwachstelle in OpenSSL
- Jeder im Internet konnte Speicher von betroffenen Servern auslesen
- Betraf ca. 17% der als Server die SSL Verbindungen anbieten

*"We attacked ourselves from outside, without leaving a trace.
... we were able steal from ourselves
user names and passwords, instant messages, emails and
business critical documents and communication."*



Eingaben des Clients nicht validiert; Annahmen auf Server Seite:

- Client teilt Server ein geheimes Passwort der Länge k_1 mit
- Clients schicken sog. *heartbeat* Nachrichten und bitten um Antwort der Länge k_2 . OpenSSL soll mit vereinbartem Passwort der Länge $k_1 = k_2$ antworten
- Schwachstelle: Server kopiert Speicher der Länge k_2 ohne Prüfung

MacOS Root Rechte



Security

Pro tip: You can log into macOS High Sierra as root with no password

Apple, this is Windows 95 bad – but there is a workaround to kill the bug

By Shaun Nichols in San Francisco 28 Nov 2017 at 20:15 149 SHARE ▼



Updated A trivial-to-exploit flaw in macOS High Sierra, aka macOS 10.13, allows users to gain admin rights, or log in as root, without a password.

- Anmeldung als 'root' ohne Passwort nach Update
- Alle Administrator Rechte
- November 2017
- Auch erstes Update löst Problem nicht vollständig

Autonome Fahrzeuge

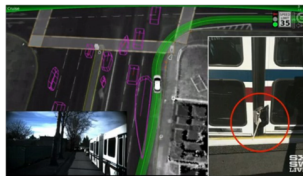
Toyota (Gaspedal klemmt)



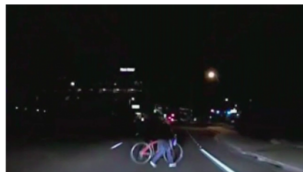
Tesla (LKW nicht erkannt)



Google (Bus gerammt)



Uber (Person überfahren)



Spirit Rover

- Abnormales Verhalten an Sol 18
- Analyse und Reparatur für 11 Sols
- Wieder normales Verhalten ab Sol 33



Flash Dateisystem stürzt ab nach (sehr kurzer) ungetesteter Folge von Operationen

- Kombination von konzeptuellen Fehlern und Konfiguration von Dateisystems und Speicherverwaltung
- Verursacht Folge von Out-of-memory, Neustart, Out-of-memory, ...
- Gelöst durch vorhandenes Notfallsystem

Knight Trading

- 1. August 2012: Bug in Software Update
- 397M Aktien in 45 Minuten gehandelt
- Schwankungen in Aktienkursen von 148 Firmen
- Z.B., Wizzard Software Corporation von \$3.50 auf \$14.76.
- Knight Capital verliert \$440 Mio. (vor Steuern).



Software startet unerwartet Legacy Funktion nach Update:

- Umgenutztes Flag in Kommunikation
- Einer von acht Servern wird vergessen
- Dort: Flag startet Funktion ein, die Algorithmen kalibriert durch das Handeln mit Aktien (gedacht für Simulationsumgebung)

Volkswagen Dieselskandal

Untersuchung / Verfahren laufen immer noch (Folie ist von 2015 ...)

- "Defeat Device" in 11M Dieselfahrzeugen
- Software schaltet Abgasnachbehandlung aus sobald erkannt wird, dass kein Laborversuch gefahren wird
- Scheinbar wurde Funktion heimlich eingebaut

Dienstag, 22. September 2015

Der Hausmeister war's! VW präsentiert Einzeltäter hinter Abgas-Affäre

f 37.7K t + 4.1K



Wolfsburg (dpo) - Im Skandal um manipulierte Abgaswerte von Dieselfahrzeugen setzt VW nach Tagen der Kritik überraschend auf volle Transparenz und legt den alleinigen Verantwortlichen offen. Demnach soll der 61-jährige Hausmeister Hans Böbner seinen Zugang zur Firmenzentrale in Wolfsburg genutzt haben, um eigenhändig Manipulationen an der Software mehrerer Millionen Fahrzeuge vorzunehmen. Inzwischen hat Böbner selbst ein erstes Geständnis abgelegt.

Ausschnitt aus: Der Postillon

- Linux Client löscht Home Verzeichnis (und mehr) bei einigen Nutzern
- Gelöschte Daten können nicht wiederhergestellt werden



pythoner commented on Jan 15, 2015

introduced here Sep 10, 2013 [indrora/steam_latest@ 21cc141](#) line 359

```
rm -rf "$STEAMROOT/**" could be evaluated as rm -rf "/" if $STEAMROOT is empty
```

but what exactly caused this? i've symlinked ~/.local/share/steam to, so i am a bit afraid to start steam :/



TcM1911 commented on Jan 15, 2015

pythoner,

I believe the issue starts on line 19:

```
# figure out the absolute path to the script being run a bit
# non-obvious, the ${0%/*} pulls the path out of $0, cd's into the
# specified directory, then uses $PWD to figure out where that
# directory lives - and all this in a subshell, so we don't affect
# $PWD

STEAMROOT="$(cd "${0%/*}" && echo $PWD)"
STEAMDATA="$STEAMROOT"
```

This probably returns as empty which mean: `rm -rf "$STEAMROOT/**"` is the same as `rm -rf "/"`.

Unsicherer Code: Inhalt von Variable nicht geprüft und Warnungen unterdrückt:

- Variable \$STEAMROOT can leerer String sein
- Programm kann `"rm -Rf /"` ausführen

Qualitätssicherung

- Was ist Qualität von Software?
- Wie kann Qualität sichergestellt werden?

Definition (Qualität)

Qualität ist der Grad, in dem eine Menge inhärenter Merkmale eines Objekts Anforderungen erfüllt.

Anforderungen (aus Beispielen): Wir wollen, dass unser Produkt ...

- ... nicht explodiert,
- ... Menschen nicht gefährdet,
- ... nicht ausfällt,
- ... Erwartungen der Benutzer erfüllt,
- ... kein Sicherheitsrisiko darstellt.

Qualitätsmerkmale für Software

- **Funktionalität:**

Korrektheit, Angemessenheit, Interoperabilität, Ordnungsmäßigkeit, Sicherheit

- **Zuverlässigkeit:**

Reife, Fehlertoleranz, Wiederherstellbarkeit

- **Benutzbarkeit:**

Verständlichkeit, Bedienbarkeit, Erlernbarkeit, Robustheit

- **Effizienz:**

Wirtschaftlichkeit, Zeitverhalten, Verbrauchsverhalten

- **Wartungsfreundlichkeit:**

Analysierbarkeit, Änderbarkeit, Stabilität, Testbarkeit

- **Übertragbarkeit:**

Anpassbarkeit, Installierbarkeit, Konformität, Austauschbarkeit

Code Qualität: Definition

Definition (Code Qualität oder Strukturelle Software Qualität)

Strukturelle Software Qualität ist der Grad, zu dem die Software die nicht-funktionalen Anforderungen erfüllt. Darunter fallen: Zuverlässigkeit / Resilienz, Performanz / Effizienz, Sicherheit, und Wartbarkeit.

Im Gegensatz dazu:

Funktionale Qualität ist der Grad in dem die Software funktionale Anforderungen erfüllt.

Fehler und Ausfälle

Definition (Ausfall)

Wenn eine Anforderung nicht (mehr) erfüllt wird.

Der Moment in dem das System explodiert ...

Definition (Fehler)

Ursache für einen Ausfall

Exception, die geworfen wurde bei 64-bit float nach 16-bit Umwandlung ...

Definition (Qualitätssicherung)

Verhinderung von Ausfällen in Produkten oder Services (durch das Verhindern oder Finden von Fehlern).

■ **Produktorientiert:**

Software und Zwischenergebnisse werden auf vorher festgelegte Qualitätsmerkmale überprüft.

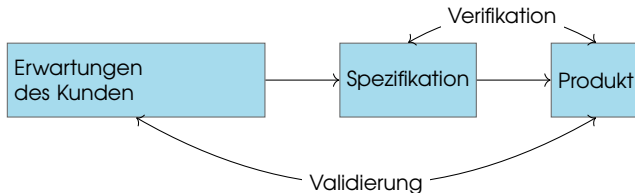
■ **Prozessorientiert:**

Methoden, Werkzeuge, Richtlinien und Standards für den Erstellungsprozess der Software

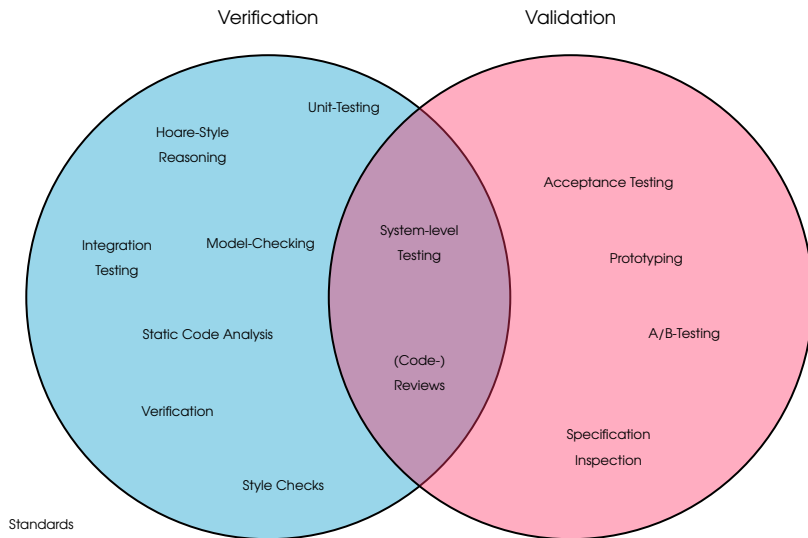
Verifikation und Validierung

Verifikation: Bauen wir das Produkt richtig?

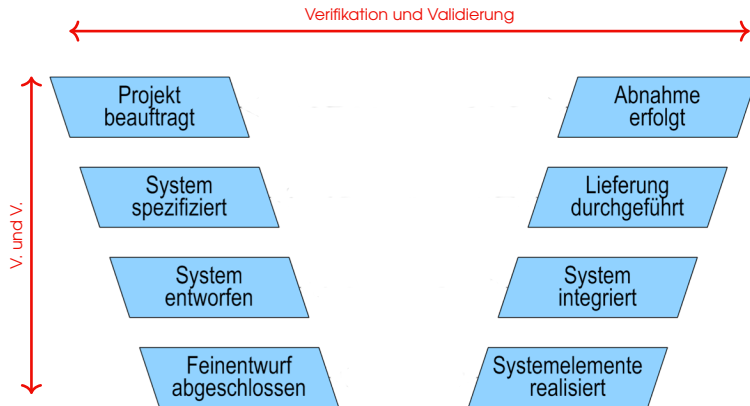
Validierung: Bauen wir das richtige Produkt?



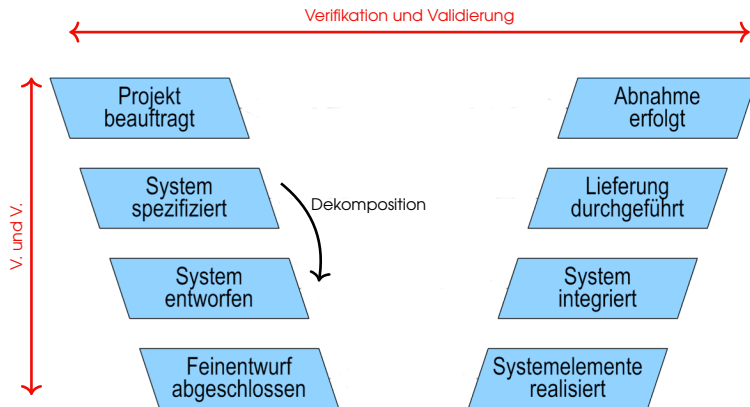
Verifikation vs. Validierung



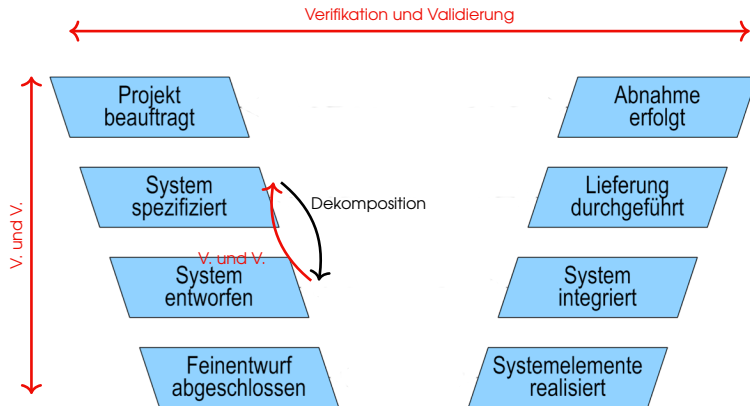
V & V im Entwicklungsprozess



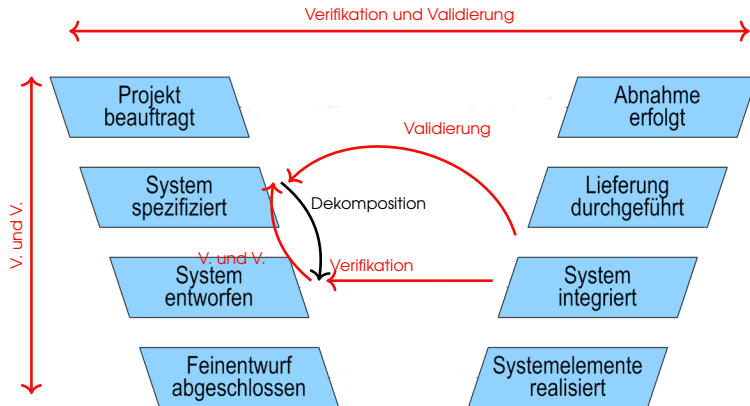
V & V im Entwicklungsprozess



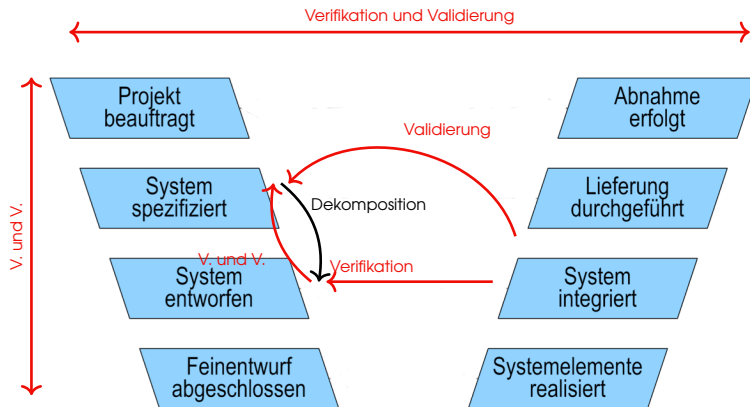
V & V im Entwicklungsprozess



V & V im Entwicklungsprozess



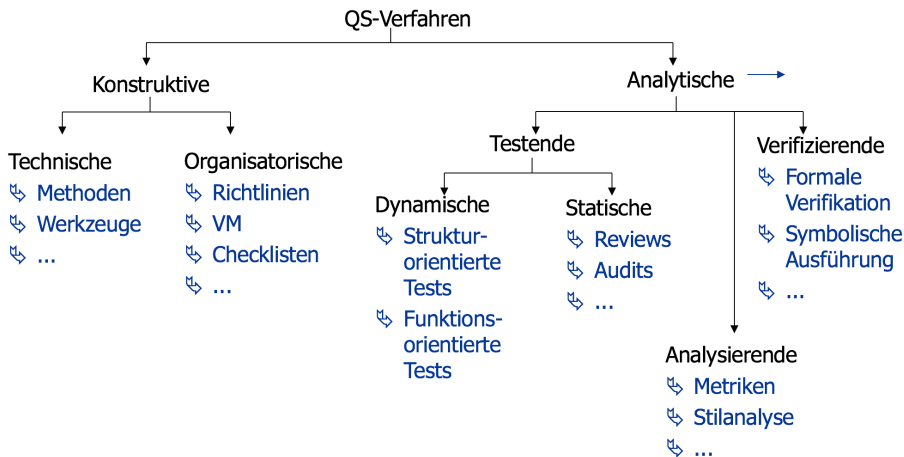
V & V im Entwicklungsprozess



⇒ **Wann?** In jedem Prozessschritt!

Jedes Artefakt des Entwicklungsprozesses kann verifiziert und validiert werden.

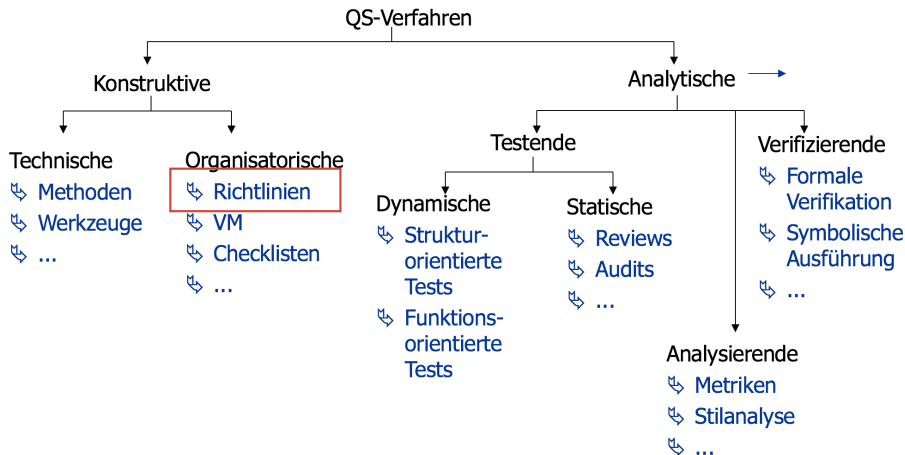
Klassifizierung von Ansätzen



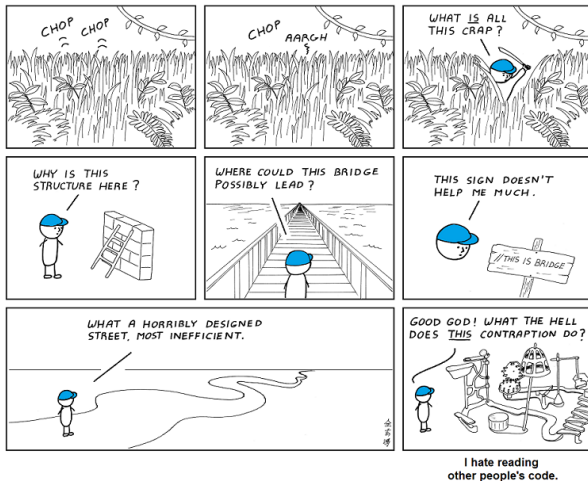
Agenda

1. Motivation und Begriffe
2. (Code) Standards
3. (Code) Reviews
4. Modellierung und Generierung
5. Verifikation: Model-Checking und Invarianten
6. Metriken und Statische Code Analyse
7. Formale Semantik and Symbolische Ausführung
8. Literatur und Standards

QS - Klassifizierung von Ansätzen



Code Qualität



Stilfragen

Beispiele

```
if(foo)
    // bar();
doSomethingElse();
```

Beispiele

```
if(foo)
    // bar();
doSomethingElse();
```

Bessere Alternativen

```
if(foo) {
    // bar();
}
```

Stilfragen

Beispiele

```
if(foo)
    // bar();
doSomethingElse();

for (int i = 1; i < a; i++) {
    for (int j = 1; j < b; j++)
        doThis();
        doThat();
}
```

Bessere Alternativen

```
if(foo) {
    // bar();
}
```

Stilfragen

Beispiele

```
if(foo)
    // bar();
doSomethingElse();

for (int i = 1; i < a; i++) {
    for (int j = 1; j < b; j++)
        doThis();
        doThat();
}
```

Bessere Alternativen

```
if(foo) {
    // bar();
}

for (int i = 1; i < a; i++) {
    for (int j = 1; j < b; j++) {
        doThis();
    }
    doThat();
}
```

Stilfragen

Beispiele

```
if(foo)
    // bar();
doSomethingElse();

for (int i = 1; i < a; i++) {
    for (int j = 1; j < b; j++)
        doThis();
        doThat();
}

boolean b = x.equals("");
```

Bessere Alternativen

```
if(foo) {
    // bar();
}

for (int i = 1; i < a; i++) {
    for (int j = 1; j < b; j++) {
        doThis();
    }
    doThat();
}
```


Stilfragen

Beispiele

```
if(foo)
    // bar();
doSomethingElse();

for (int i = 1; i < a; i++) {
    for (int j = 1; j < b; j++)
        doThis();
        doThat();
}

boolean b = x.equals("");
```

Bessere Alternativen

```
if(foo) {
    // bar();
}

for (int i = 1; i < a; i++) {
    for (int j = 1; j < b; j++) {
        doThis();
    }
    doThat();
}

boolean b = "".equals(x);
```

Stilfragen

Beispiele

```
if(foo)
    // bar();
doSomethingElse();

for (int i = 1; i < a; i++) {
    for (int j = 1; j < b; j++)
        doThis();
        doThat();
}

boolean b = x.equals("");

int doIt(int x) {...}
```

Bessere Alternativen

```
if(foo) {
    // bar();
}

for (int i = 1; i < a; i++) {
    for (int j = 1; j < b; j++) {
        doThis();
    }
    doThat();
}

boolean b = "".equals(x);
```

Stilfragen

Beispiele

```
if(foo)
    // bar();
doSomethingElse();

for (int i = 1; i < a; i++) {
    for (int j = 1; j < b; j++)
        doThis();
        doThat();
}

boolean b = x.equals("");

int doIt(int x) {...}
```

Bessere Alternativen

```
if(foo) {
    // bar();
}

for (int i = 1; i < a; i++) {
    for (int j = 1; j < b; j++) {
        doThis();
    }
    doThat();
}

boolean b = "".equals(x);

/**
 * computer a random prime
 * number between
 * 0 and upperBound.
 */
int findPrime(int upperBound) {...}
```

Coding Standards

Definition (Coding Standard)

Ein Coding Standard ist eine Menge von Regeln, die einen bestimmten Programmierstil vorschreiben. Regeln beinhalten üblicherweise:

Einrückung, Organisation, Deklaration, Fehlerbehandlung, Verzweigende Anweisungen, Test Abdeckung, . . .

Beispiele:

- Variablen werden oben in der Klasse deklariert.
- Es soll eine Test-Abdeckung von 80% (Verzweigungen) geben
- Alle Ausnahmen müssen behandelt werden

Coding Standards adressieren Software Qualität

Wartbarkeit

- Dokumentation aller Methoden.
- Klammerung: 1TBS.

Zuverlässigkeit

- Alle Ausnahmen müssen behandelt werden.
- Alle Eingaben müssen validiert werden.

Sicherheit

- SQL Anfragen werden nicht manuell zusammengebaut.
- Code muss zu einem gewissen Grad getestet sein.

Effizienz

- Nutze `StringBuilder` anstelle von Konkatination.

Coding Standards adressieren Software Qualität

Wartbarkeit

- Dokumentation aller Methoden.
- Klammerung: 1TBS.

Zuverlässigkeit

- Alle Ausnahmen müssen behandelt werden.
- Alle Eingaben müssen validiert werden.

Sicherheit

- SQL Anfragen werden nicht manuell zusammengebaut.
- Code muss zu einem gewissen Grad getestet sein.

Effizienz

- Nutze `StringBuilder` anstelle von Konkatination.

Beispiel: NASA JPL

Regeln aus Projekt des Jet Propulsion Laboratory (JPL) für sicherheitskritische Software (für Code in C).

- ① Restrict all code to very simple control flow constructs – do not use goto ...
- ② All loops must have a fixed upper-bound. It must be trivially possible for a checking tool to prove statically that a preset upper-bound on the number of iterations of a loop cannot be exceeded. If the loop-bound cannot be proven statically, the rule is considered violated.
- ③ Do not use dynamic memory allocation after initialization.
- ④ No function should be longer than what can be printed on a single sheet of paper in a standard reference format with one line per statement and one line per declaration. Typically, this means no more than about 60 lines of code per function.
- ⑤ The assertion density of the code should average to a minimum of two assertions per function. Assertions are used to check for anomalous conditions that should never happen in real-life executions. ...
- ⑥ The return value of non-void functions must be checked by each calling function, and the validity of parameters must be checked inside each function.
- ⑦ All code must be compiled, from the first day of development, ... All code must be checked daily with at least one, ..., state-of-the-art static source code analyzer ...
- ⑧ ...

Coding Standards:

Diskussion und Zusammenfassung

Diskussion:

- Regeln müssen spezifisch sein: Z.b. was bedeutet "Don't use Hacks"?
- Weniger ist mehr: Niemand kann 50 Regeln behalten,
- Kontrolle der Einhaltung ist essenziell,
- Automatisierung / Werkzeuge helfen (⇒ Statische Code Analyse).

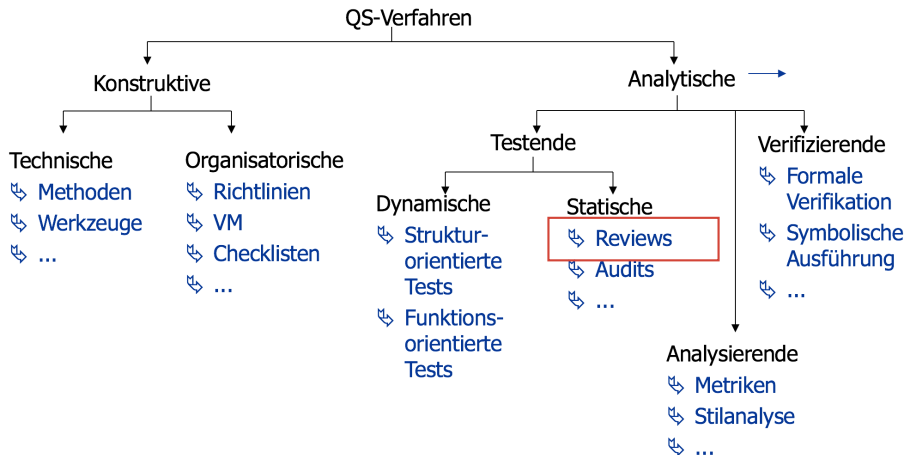
Zusammenfassung:

- Regeln / Anweisungen für Format und Eigenschaften von Code
- vgl. "Codex"
- Ziele: Hohe Qualität von Code, keine sogenannten "Code Smells"
- Hohe Qualität: Wartbar, sicher, effizient, ...

Agenda

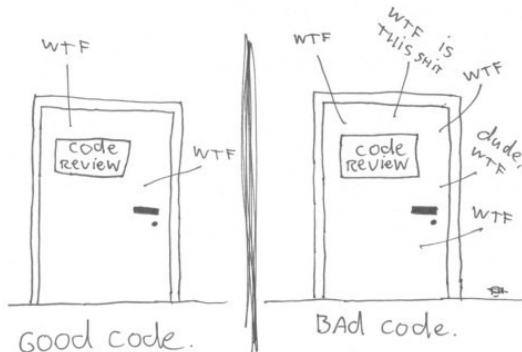
1. Motivation und Begriffe
2. (Code) Standards
3. (Code) Reviews
4. Modellierung und Generierung
5. Verifikation: Model-Checking und Invarianten
6. Metriken und Statische Code Analyse
7. Formale Semantik and Symbolische Ausführung
8. Literatur und Standards

QS - Klassifizierung von Ansätzen



Code Reviews

The ONLY valid measurement
OF code QUALITY: WTFs/minute



(c) 2008 Focus Shift/OSNews/Thom Holwerda - <http://www.osnews.com/comics>

Code Reviews

Definition (Code Review)

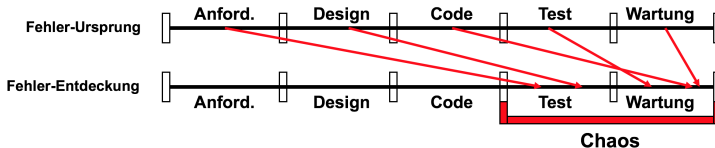
Ein Code Review ist eine systematische Untersuchung und Bewertung von Code — oftmals von anderen Personen als dem Autor des Codes.

Code Reviews helfen dabei ...

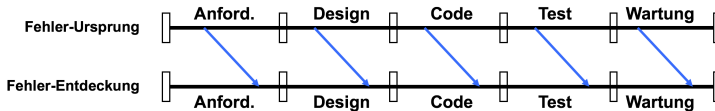
- ... Fehler früh zu finden,
- ... die Qualität des Codes zu erhöhen und sicherzustellen,
- ... geteiltes Verständnis herzustellen.

Code Reviews: Fehler früh finden

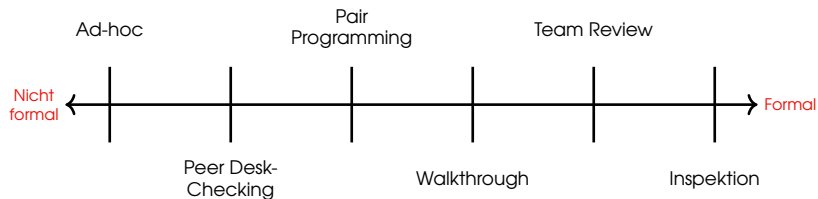
Ohne Reviews



Mit Reviews (ideal)



Code Reviews: Methoden



Code Reviews: Methoden (1)

Ad-hoc

- Kollegen um Hilfe bitten
- Risiko: Nur konkretes Problem wird angeschaut

Peer Desk-Checking

- Reviewer schaut Code am Platz des Kollegen oder auf Papier an

Pair-Programming

- Programmieren zu zweit. Eine Person schreibt, eine prüft.
- Eine Technik des "eXtreme Programming"

Code Reviews: Methoden (2)

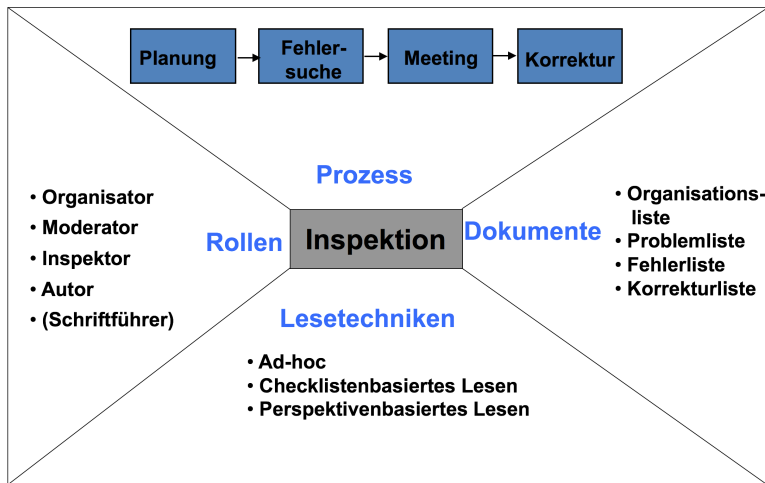
Walkthrough

- Autor präsentiert Code in Termin vor Reviewern
- Kein Vorgeschriebenes Vorgehen
- Risiko: Autor kontrolliert Fokus des Reviews

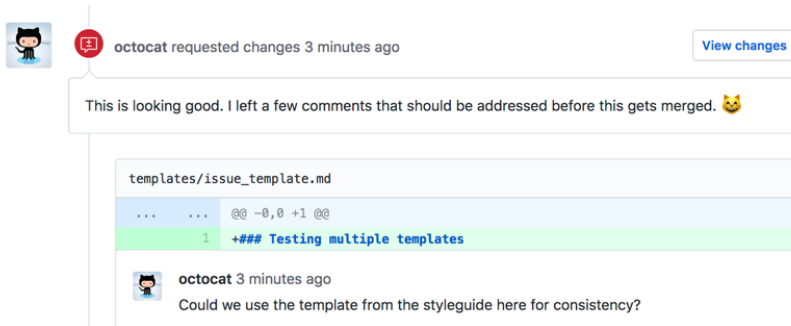
Team-Review

- Mehrere Personen produzieren Review Berichte
- Ergebnisse werden in Termin mit Autor des Codes besprochen

Code Reviews: Inspektion



Beispiel: Git Pull Requests



The screenshot shows a GitHub pull request interface. At the top, the octocat avatar is next to a red speech bubble icon and the text "octocat requested changes 3 minutes ago". A blue button labeled "View changes" is in the top right. Below this is a review comment box containing the text "This is looking good. I left a few comments that should be addressed before this gets merged." followed by a cat face emoji. Underneath the comment is a code diff for the file "templates/issue_template.md". The diff shows a line change from "... @@ -0,0 +1 @@" to "1 +### Testing multiple templates". Below the code diff is another comment from octocat, dated "3 minutes ago", asking "Could we use the template from the styleguide here for consistency?".

octocat requested changes 3 minutes ago [View changes](#)

This is looking good. I left a few comments that should be addressed before this gets merged. 🐱

templates/issue_template.md

```
... @@ -0,0 +1 @@  
1 +### Testing multiple templates
```

octocat 3 minutes ago
Could we use the template from the styleguide here for consistency?

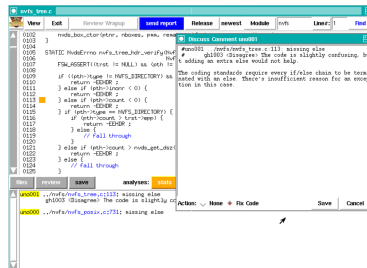
Web-basiertes Review von Pull Requests

- Kommentare möglich an einzelnen Zeilen im Code
- Kommentare müssen adressiert werden

Beispiel: NASA JPL

Werkzeug-gestützte Inspektion:

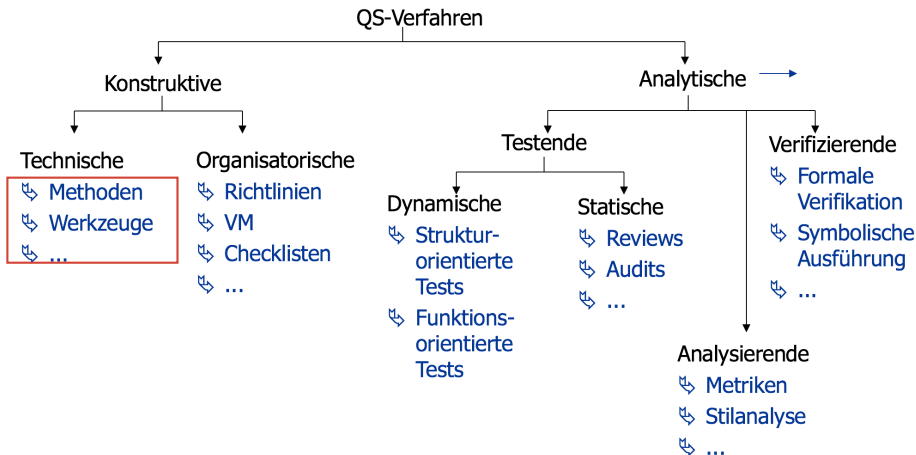
- Werkzeug für Review / Antworten
- Ergebnisse von automatisierten Analysen im Review
- Diskussion wird gespeichert
- Kurzes Treffen (60-90 min) für jedes Modul
- Kommentare werden im Treffen diskutiert



Agenda

1. Motivation und Begriffe
2. (Code) Standards
3. (Code) Reviews
- 4. Modellierung und Generierung**
5. Verifikation: Model-Checking und Invarianten
6. Metriken und Statische Code Analyse
7. Formale Semantik and Symbolische Ausführung
8. Literatur und Standards

QS - Klassifizierung von Ansätzen

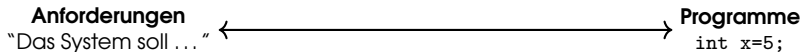


Werkzeugbau im Software Engineering



<https://flic.kr/p/fkmB7T>, (CC BY 2.0)

Semantische Lücke



Fachlichkeit vs. Technik

- Abstraktionsniveau derzeitiger Entwicklungsansätze zu niedrig
- Umsetzung fachlicher Basiskonzepte dominiert durch Technik
- Anwendungsentwickler benötigen umfangreiches technisches Wissen statt sich auf fachliche Anwendungsdomäne zu konzentrieren

Fehlende Durchgängigkeit

- Fehlender Zusammenhang zwischen Anforderungen, Dokumentation und Code
- Dokumentation nicht aktualisiert
→ nahezu nutzlos.
- Umsetzung von Anforderungen nicht nachvollziehbar
→ Abnahme schwierig

Semantische Lücke (contd.)



Fachlichkeit vs. Technik

Bedarf nach höherem
Abstraktionsniveau

→

Modelle Abstrahieren und fokussieren
auf das Wesentliche

Fehlende Durchgängigkeit

Durchgängigere Auswahl der
Ausdrucksmittel

→

Semantische Brücke von fachlicher
Problemwelt in technische Lösungswelt

Von Anforderungen zu Programmen

Anforderungen

Integration von
Aspekten

Konkretisierung von
Lösungen

Modellierungssprachen



Modellgetriebene Softwareentwicklung

Aspekte von
Problemen

Modellierungssprachen



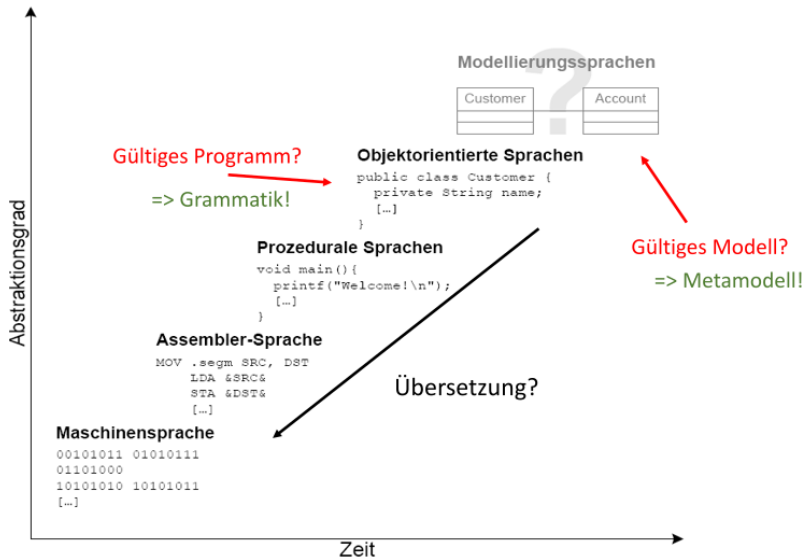
Domänenspezifische Sprachen

Abstraktion von
Implementierungsdetails

Programme



Entwicklung von Modellierungssprachen



Korrektheit von Modellen

Syntaktisch

- Ist es ein gültiges Modell?
- Ist das Modell korrekt gezeichnet?

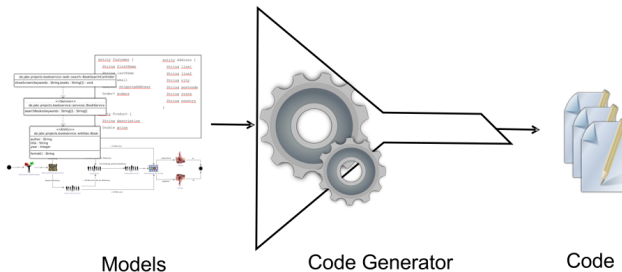
Voraussetzung für Übersetzung

Semantisch

- Ist das Modell korrekt?
- Erfüllt das Modell die Anforderungen?

Voraussetzung für Verwendung

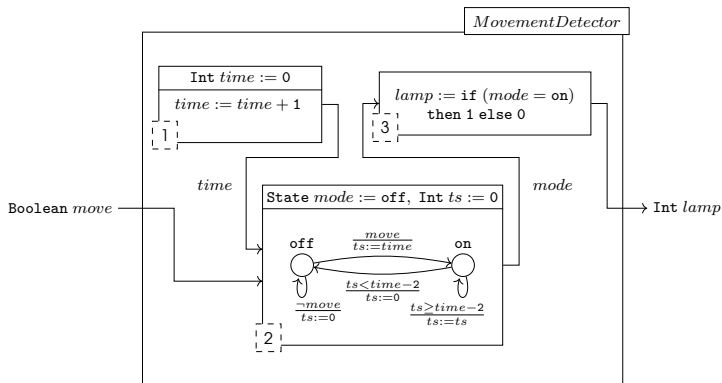
Übersetzung von Modellen



Trennung fachlicher und technischer Aspekte:

- Fachliche Lösung im Modell
- Technische Aspekte der Lösung im Code-Generator

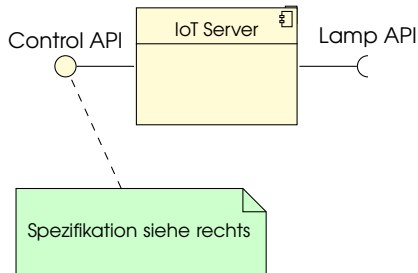
Beispiel: Remote Home Control



Beispiele für Fragen:

- Geht das Licht immer an, wenn sich etwas bewegt?

Beispiel: Schnittstelle an Komponente



Control API:

- `listLamps() → List<Lamp>`
- `getStatus(Lamp) → [on:Bool,err:Int]`
- `turnOn(Lamp) → [on:Bool,err:Int]`
- `turnOff(Lamp) → [on:Bool,err:Int]`

Anforderung: nach `turnOn(Lamp)` gilt $(on \vee (err > 0))$

Variablen / Typen

Typen:

- `Int`: Ganze Zahlen
- `Boolean`: Boolesche Werte $\{0, 1\}$
- Aufzählungen. Z.B. Typ `State` mit Werten $\{\text{on}, \text{off}\}$

Variablen:

- Menge X von getypten Variablen

Getypete Ausdrücke

Für einen fixen Aufzählungstyp, z.B. **State**:

$e \in \mathbf{EExp}$ (Aufzählungs-Ausdrücke)

$e ::= z \mid c$

$z \in \mathbf{Var}$ Variable vom Aufzählungstyp **State**

$c \in \mathbf{State}$ konkreter Wert vom passenden Typ

Getypte Ausdrücke

$a \in \mathbf{AExp}$ (Arithmetische Ausdrücke)

$a ::= x \mid \mathbf{n} \mid (a_1 \text{ } op_a \text{ } a_2)$

$b \in \mathbf{BExp}$ (Boolesche Ausdrücke)

$b ::= y \mid \mathbf{true} \mid \mathbf{false} \mid \neg b \mid (b_1 \text{ } op_b \text{ } b_2) \mid (a_1 \text{ } op_r \text{ } a_2) \mid (z = c)$

$x \in X$ Variable Typ **Int**

$y \in X$ Variable Typ **Boolean**

$z \in X$ Variable Typ **State**

$\mathbf{n}$ Literal für $n \in \mathbb{Z}$

$c \in \mathbf{State}$

$op_a \in \mathbf{Op}_a$ arithmetische Op.: $+$, $-$

$op_b \in \mathbf{Op}_b$ Boolesche Op.: $\wedge$, $\vee$

$op_r \in \mathbf{Op}_r$ Relationale Op.: $>$, $<$, $=$

Beispiel

Variablen:

- $X = \{x_1, x_2\}$

Typen:

- x_1 von Typ `Int`
- x_2 von Typ `State` mit Werten `{on, off}`

Ausdruck:

$$((x_2 = \text{off}) \wedge (x_1 > 50))$$

Belegungen

Belegung

Eine Belegung v von X ist eine Typ-konsistente Zuweisung von Werten zu Variablen in X . v ist eine Funktion und für $x \in X$ ist $v(x)$ der Wert mit dem x in v belegt ist.

- Typ-konsistent: $v(x)$ gehört zu Typ von x .
- Z.B., $v(x) \in \mathbb{Z}$ wenn x von Typ `Int`.

Sei V_X die Menge aller möglichen typ-konsistenten Belegungen von X .

Beispiel

Variablen:

- $X = \{x_1, x_2\}$

Typen:

- x_1 von Typ Int
- x_2 von Typ State mit Werten {on, off}

Belegung:

- v_1 mit $v_1(x_1) = 101$ und $v_1(x_2) = \text{on}$

Auswertung von Ausdrücken

- Set von Variablen X
- Ausdrücke aus **Exp** über Variablen aus X
- Fixer Aufzählungstyp z.B., State mit Werten $\{\text{on}, \text{off}\}$

Auswertung:

- Funktion $\alpha_X : \mathbf{AExp} \times V_X \mapsto \mathbb{Z}$
- Funktion $\beta_X : \mathbf{BExp} \times V_X \mapsto \{0, 1\}$
- Funktion $\gamma_X : \mathbf{EExp} \times V_X \mapsto \text{State}$
- Belegung v von X

Auswertung von Aufzählungsausdrücken

Atome:

- $\gamma_X[\mathbf{on}, v] = \mathbf{on}$
- $\gamma_X[\mathbf{off}, v] = \mathbf{off}$
- $\gamma_X[z, v] = v(z)$

Auswertung von Booleschen Ausdrücken

Atome:

- $\beta_X[\mathbf{true}, v] = 1$
- $\beta_X[\mathbf{false}, v] = 0$
- $\beta_X[y, v] = v(y)$

Negation:

- $\beta_X[(\neg b), v] = 1$ gdw. $\beta_X[b, v] = 0$ (sonst 0)

Komposition:

- $\beta_X[(b_1 \wedge b_2), v] = 1$ gdw. $\beta_X[b_1, v] = 1$ und $\beta_X[b_2, v] = 1$ (sonst 0)
- $\beta_X[(b_1 \vee b_2), v] = 0$ gdw. $\beta_X[b_1, v] = 0$ und $\beta_X[b_2, v] = 0$ (sonst 1)

Auswertung von arithmetischen Ausdrücken

Atome:

- $\alpha_X[\mathbf{n}, v] = n$
- $\alpha_X[x, v] = v(x)$

Komposition:

- $\alpha_X[(a_1 + a_2), v] = \alpha_X[a_1, v] + \alpha_X[a_2, v]$
- $\alpha_X[(a_1 - a_2), v] = \alpha_X[a_1, v] - \alpha_X[a_2, v]$

Wir trennen hier nicht formal sauber zwischen Syntax und Semantik von $+$, $-$!

Boolesche Kombination:

Aufzählung:

- $\beta_X[(z = c), v] = 1$ gdw. $\gamma_X[z, v] = \gamma_X[c, v]$ (sonst 0)

Arithmetisch:

- $\beta_X[(a_1 \circ a_2), v] = 1$ gdw. $\alpha_X[a_1, v] \circ \alpha_X[a_2, v]$

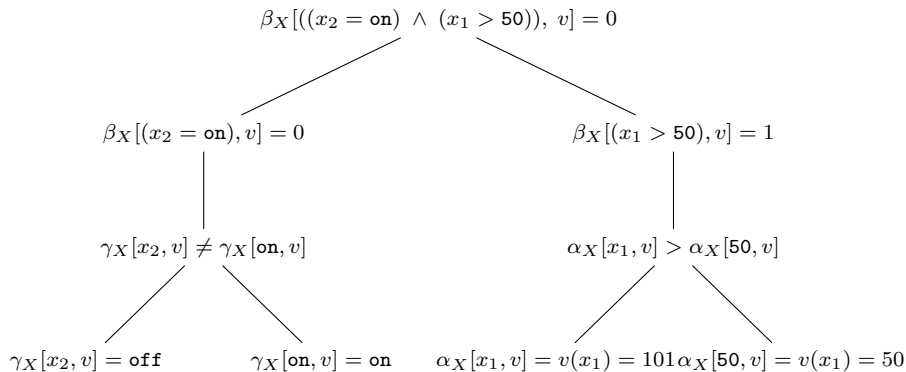
Für $\circ = \{<, =, >\}$

Beispiel

- Ausdruck: $((x_2 = \text{on}) \wedge (x_1 > 50))$
- Belegung: v mit $v(x_1) = 101$ und $v(x_2) = \text{off}$

Beispiel

- Ausdruck: $((x_2 = \text{on}) \wedge (x_1 > 50))$
- Belegung: v mit $v(x_1) = 101$ und $v(x_2) = \text{off}$



Umgang mit Variablen

Ausdrücke mit Variablen:

- Wir schreiben $\varphi[\mathbf{x}]$ um anzuzeigen, dass der Ausdruck φ über einer Menge $\mathbf{x}$ von Variablen formuliert ist

Beispiel:

- Für $\varphi = ((x_2 = \text{on}) \wedge (x_1 > 50))$
- $\varphi[\mathbf{x}]$ mit $\mathbf{x} = \{x_1, x_2\}$

Partielle Belegungen:

- Sei $\mathbf{z} \subset \mathbf{x}$ und $x \in \mathbf{x} \setminus \mathbf{z}$. Wir schreiben $v_{\mathbf{z}}[x \mapsto d]$ für die Belegung $v_{\mathbf{z} \cup \{x\}}$ mit
$$v_{\mathbf{z} \cup \{x\}}(y) = v_{\mathbf{z}}(y) \text{ für } y \neq x \text{ und}$$
$$v_{\mathbf{z} \cup \{x\}}(y) = d \text{ für } y = x$$
$$(\text{jeweils } y \in \mathbf{z} \cup \{x\}).$$

Erfüllbarkeit

Frage: Ist Ausdruck $\varphi[\mathbf{x}] \in \mathbf{BExp}$ erfüllbar

Existiert eine Belegung v von $\mathbf{x}$ so dass $\beta_{\mathbf{x}}[\varphi, v] = 1$?

Wir beschränken uns in SWK ab jetzt auf rein Boolesche Ausdrücke in konjunktiver Normalform (KNF)

$t \in \mathbf{BCNF}$ (rein Boolesche Ausdrücke in KNF)

$t ::= (c) (\wedge (c))^*$

$c ::= l (\vee l)^*$ (Clauses)

$l ::= y \mid \neg y$ (Literals)

$y \in X$ Variable Typ Boolean

$op_b \in \mathbf{Op_b}$ Boolesche Op.: $\wedge, \vee$

Wir verzichten für Lesbarkeit auf vollständige Klammerung

Beispiel: Erfüllbarkeit

Variablen:

- $X = \{x_1, x_2, x_3\}$ von Typ Int

Ausdruck $(x_1 \vee x_2) \wedge (\neg x_2 \vee x_3) \wedge (x_2 \vee \neg x_3)$ ist erfüllbar:

- z.B. v_1 mit $v_1(x_1) = 1$ und $v_1(x_2) = 0$ und $v_1(x_3) = 0$

Ausdruck $(x_1 \wedge \neg x_1)$ ist nicht erfüllbar.

Erfüllbarkeit und Widerspruch von Anforderungen

Sei $\varphi[\mathbf{x}], \psi[\mathbf{x}] \in \mathbf{BCNF}$.

- **Erfüllbarkeit (SAT):**

φ ist erfüllbar, wenn Belegung $v_{\mathbf{x}}$ mit $\beta_{\mathbf{x}}[\varphi, v_{\mathbf{x}}] = 1$ existiert.

- **Widerspruch:**

φ und ψ sind widersprüchlich, wenn $(\varphi \wedge \psi)$ nicht erfüllbar ist.

- **Implikation (Entailment):**

φ impliziert ψ , wenn $(\varphi \wedge \neg\psi)$ nicht erfüllbar ist.

$(\varphi \wedge \neg\psi)$ ist nicht in KNF!

Beachte:

Wir verlangen, dass φ und ψ über den gleichen Variablen definiert sind!

Wiederholung: DPLL-Algorithmus

- Rekursiver Algorithmus für die Entscheidung der Erfüllbarkeit von Booleschen Ausdrücken in KNF.
- DPLL: Davis, Putnam, Logemann, Loveland (1962)
- Worst-Case Kosten: $O(2^n)$ für Literalen über n Variablen (jede mögliche Belegung)
- Im Durchschnitt (in der Praxis) oft performant
- Basis für bessere Algorithmen für das Erfüllbarkeitsproblem

Wiederholung: DPLL-Algorithmus (Beobachtungen)

Pure Literale

- Wenn im Ausdruck nur x vorkommt kann x mit 1 belegt werden
- Wenn im Ausdruck nur $\neg x$ vorkommt kann x mit 0 belegt werden

Unit Klauseln

Sei $\mathbf{z} \subset \mathbf{x}$ und $v_{\mathbf{z}}$, so dass in Klausel c **genau ein** Literal zu einer Variable aus $\mathbf{x} \setminus \mathbf{z}$ vorkommt

- Wenn in c Literal x vorkommt muss x mit 1 belegt werden
- Wenn in c Literal $\neg x$ vorkommt muss x mit 0 belegt werden

Wiederholung: DPLL-Algorithmus

Input: Formula $\varphi \in \mathbf{BCNF}$ mit Klauselmenge C über Variablen $\mathbf{x}$

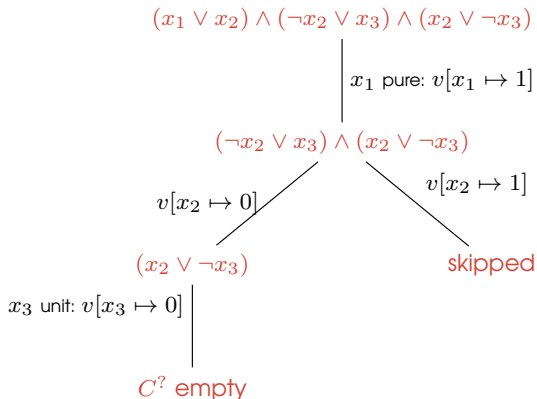
Output: *true* wenn φ erfüllbar ist, sonst *false*

```
1: procedure DPLL( $C, v_{\mathbf{z}}$  für  $\mathbf{z} \subseteq \mathbf{x}$ )
2:    $C^- \leftarrow \{c \in C : c[v_{\mathbf{z}}] \rightarrow \text{false}\}$ 
3:   if  $C^-$  is not empty then
4:     return false
5:   end if
6:    $C^? \leftarrow C \setminus \{c \in C : \text{true} \rightarrow c[v_{\mathbf{z}}]\}$ 
7:   if  $C^?$  is empty then
8:     return true
9:   end if
10:  if unit clause for literal  $x$  or  $\bar{x}$  in  $C^?$  then
11:    return DPLL( $C^?, v_{\mathbf{z}}[x \mapsto b]$ ) with  $b = 1$  for  $x$  and  $b = 0$  for  $\bar{x}$ 
12:  end if
13:  if pure literal  $x$  or  $\bar{x}$  in  $C^?$  then
14:    return DPLL( $C^?, v_{\mathbf{z}}[x \mapsto b]$ ) with  $b = 1$  for  $x$  and  $b = 0$  for  $\bar{x}$ 
15:  end if
16:  Pick  $x$  from  $\mathbf{x} \setminus \mathbf{z}$ 
17:  return DPLL( $C^?, v_{\mathbf{z}}[x \mapsto 0]$ )  $\vee$  DPLL( $C^?, v_{\mathbf{z}}[x \mapsto 1]$ )
18: end procedure
```

Beispiel: DPLL-Algorithmus

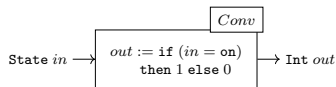
$$(x_1 \vee x_2) \wedge (\neg x_2 \vee x_3) \wedge (x_2 \vee \neg x_3)$$

Beispiel: DPLL-Algorithmus



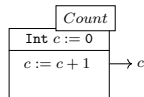
- Testet eine Belegung anstelle von $2^3 = 8!$

Modellierung des Verhaltens von Komponenten



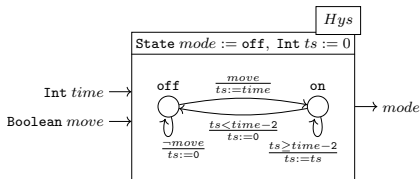
Ziel 1:

- Formale Spezifikation für Komponenten und Komposition (Anknüpfend an Vorlesung über Architektur)



Ziel 2:

- Basis für Model Checking und Invariantenbeweise



Umgang mit Variablen

Ausdrücke mit Variablen:

- Wir schreiben $\varphi[\mathbf{x}]$ um anzuzeigen, dass der Ausdruck φ über einer Menge $\mathbf{x}$ von Variablen formuliert ist
- Mit $\mathbb{F}_{\mathbf{x}}$ bezeichnen wir Menge aller möglichen logischen Formeln aus $\mathbf{Exp}$ mit Variablen $\mathbf{x}$. Wenn $\varphi[\mathbf{y}] \in \mathbb{F}_{\mathbf{x}}$ dann $\mathbf{y} \subseteq \mathbf{x}$.

Apostrophierte Variablen und Ersetzung in Termen und Belegungen:

- Für $\mathbf{x} = \{x_1, \dots, x_k\}$ sei $\mathbf{x}' = \{x'_1, \dots, x'_k\}$ und $\mathbf{x}^i = \{x_1^i, \dots, x_k^i\}$
- Wir schreiben $\varphi[\mathbf{x}'/\mathbf{x}]$ für den Ausdruck, der aus φ durch Ersetzen der Variablen in $\mathbf{x}$ durch jene in $\mathbf{x}'$ entsteht
- Wir schreiben $v[\mathbf{x}'/\mathbf{x}]$ für die Belegung, die aus v durch Ersetzen der Variablen in $\mathbf{x}$ durch jene in $\mathbf{x}'$ entsteht

Beispiel:

- Für $\mathbf{x} = \{x_1, x_2\}$ und $\varphi = ((x_2 = \text{on}) \wedge (x_1 > 50))$ und $v_{\mathbf{x}} = \{x_1 \mapsto 1, x_2 \mapsto \text{off}\}$
- ist $\varphi[\mathbf{x}'/\mathbf{x}] = ((x'_2 = \text{on}) \wedge (x'_1 > 50))$ und $v_{\mathbf{x}}[\mathbf{x}'/\mathbf{x}] = \{x'_1 \mapsto 1, x'_2 \mapsto \text{off}\}$

Belegungen, Modelle und Erfüllbarkeit

Belegung:

- $v_{\mathbf{x}} : \mathbf{x} \mapsto \mathbb{N} \cup \{0, 1\} \cup \{\text{on}, \text{off}\}$ ist eine typ-korrekte Belegung der Variablen in $\mathbf{x}$
- $V_{\mathbf{x}}$ ist die Menge aller möglichen typ-korrekten Belegungen der Variablen in $\mathbf{x}$
- Für $\mathbf{x} \cap \mathbf{y} = \emptyset$ sei $v_{\mathbf{x}} \circ v_{\mathbf{y}}$ eine Belegung von $\mathbf{x} \cup \mathbf{y}$ mit

$$v_{\mathbf{x}} \circ v_{\mathbf{y}}(x) =_{def} \begin{cases} v_{\mathbf{x}}(x) & \text{für } x \in \mathbf{x} \\ v_{\mathbf{y}}(x) & \text{für } x \in \mathbf{y} \end{cases}$$

Modell:

- Belegung $v_{\mathbf{x}}$ ist ein Modell für $\varphi[\mathbf{x}] \in \mathbf{BExp}$ wenn $\beta_{\mathbf{x}}[\varphi, v_{\mathbf{x}}] = 1$
- Wir schreiben $v_{\mathbf{x}} \models \varphi[\mathbf{x}]$

Erfüllbarkeit:

- Ausdruck $\varphi[\mathbf{x}] \in \mathbf{BExp}$ ist erfüllbar wenn ein Modell für $\varphi[\mathbf{x}]$ existiert

Definition (Synchrone Reaktive Komponente)

Eine Synchrone Reaktive Komponente C ist ein Tupel $\langle i, s, o, I, \Gamma \rangle$ mit

- einer Menge i von Eingabevariablen,
- einer Menge s von internen Variablen,
- einer Menge o von Ausgabevariablen,
- einer initialen Konfiguration, beschrieben durch $I \in \mathbb{F}_s$
- einer Transitionsrelation $\rightarrow \subseteq V_s \times V_i \times V_o \times V_s$, beschrieben durch $\Gamma \in \mathbb{F}_{s \cup i \cup o \times s'}$.

Annahmen:

- Eine interne Variable kann auch in der Ausgabe vorkommen. Aber $i \cap s = \emptyset$ und $i \cap o = \emptyset$
- Komponenten müssen keine Eingabe, Zustand, Ausgabe haben, also i, s oder o können leer sein.

Verhalten von Komponenten (Transitionen)

Komponente $C = \langle \mathbf{i}, \mathbf{s}, \mathbf{o}, I, \Gamma \rangle$

Semantik

Initialisierung:

- Belegung $v_s \in V_s$ ist eine initiale Konfiguration von C wenn $v_s \models I$

Transitionen:

- Tupel $(v_s, v_i, v_o, v'_s) \in \rightarrow$ wenn $v_s \circ v_i \circ v_o[\mathbf{o}'/\mathbf{o}] \circ v'_s[\mathbf{s}'/\mathbf{s}] \models \Gamma$
- Wir schreiben $v_s \xrightarrow{v_i/v_o} v'_s$

Annahmen:

- Variablen aus $\mathbf{s}$ müssen nicht (eindeutig) initialisiert werden
- Komponente kann nicht-deterministisch sein ($\rightarrow$ ist eine Relation).
- Komponenten können immer schalten:
Für jedes Paar (v_s, v_i) existiert min. eine Transition $(v_s, v_i, v_o, v'_s) \in \rightarrow$

Verhalten von Komponenten (Läufe)

Semantik

Komponente $C = \langle \mathbf{i}, \mathbf{s}, \mathbf{o}, I, \Gamma \rangle$

Lauf:

- (Unendliche) Folge von Transitionen $v_s^0 \xrightarrow{v_i^1/v_o^1} v_s^1 \xrightarrow{v_i^2/v_o^2} v_s^2 \xrightarrow{v_i^3/v_o^3} \dots$ mit $v_s^0 \models I$

Verhalten von C :

- Menge aller Läufe

Anmerkung:

- Wir beschäftigen uns hier nur mit endlichen Präfixen von Läufen

Beispiel

Komponente $Delay = \langle \mathbf{i}, \mathbf{s}, \mathbf{o}, I, \Gamma \rangle$ mit

- $\mathbf{i} = \{i_1\}, \mathbf{s} = \{s_1\}, \mathbf{o} = \{o_1\}$ und i_1, s_1, o_1 vom Typ Int
- $I = (s_1 = 0)$
- $\Gamma = (o'_1 = s_1 \wedge s'_1 = i_1)$

Transitionen in $\rightarrow$ (Beispiele):

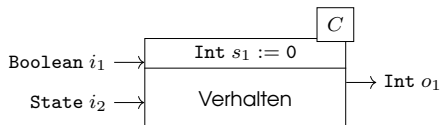
- $\{s_1 \mapsto 0\} \xrightarrow{\{i_1 \mapsto 0\} / \{o_1 \mapsto 0\}} \{s_1 \mapsto 0\}$
- $\{s_1 \mapsto 0\} \xrightarrow{\{i_1 \mapsto 1\} / \{o_1 \mapsto 0\}} \{s_1 \mapsto 1\}$
- $\{s_1 \mapsto 2\} \xrightarrow{\{i_1 \mapsto 3\} / \{o_1 \mapsto 2\}} \{s_1 \mapsto 3\}$
- $\{s_1 \mapsto 3\} \xrightarrow{\{i_1 \mapsto 5\} / \{o_1 \mapsto 3\}} \{s_1 \mapsto 5\}$

Lauf (Anfang):

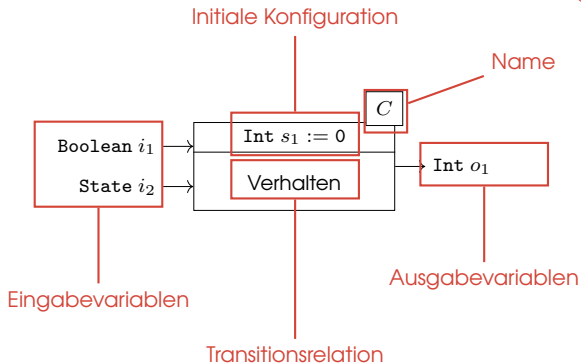
Var.	t_0	t_1	t_2	t_3	t_4
i_1	1	2	5	5	3
s_1	0	1	2	5	5
s'_1	1	2	5	5	3
o'_1	0	1	2	5	5

Konkrete Syntax

Konkrete
Syntax

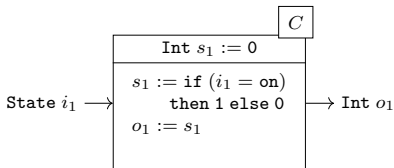


Konkrete Syntax



Kombinatorische Komponente

Konkrete Syntax:



Verhalten:

- Liste von Zuweisungen $x := e$ für $x \in (\mathbf{s} \cup \mathbf{o})$ und $e \in \mathbb{F}_{\mathbf{s} \cup \mathbf{i} \cup \mathbf{s}'}$
- Eine Zuweisung pro Variable
- Sequenzielle Ausführung

Abstrakte Syntax:

$C = \langle \mathbf{i}, \mathbf{s}, \mathbf{o}, \mathbf{I}, \Gamma \rangle$ mit

$\mathbf{i} = \{i_1\}$, $\mathbf{s} = \{s_1\}$, $\mathbf{o} = \{o_1\}$,

$\mathbf{I} = (s_1 = 0)$ und

$\Gamma = (s'_1 = \text{if } (i_1 = \text{on}) \text{ then } 1 \text{ else } 0) \wedge$
 $(o'_1 = s'_1)$

Übersetzung der Zuweisungen:

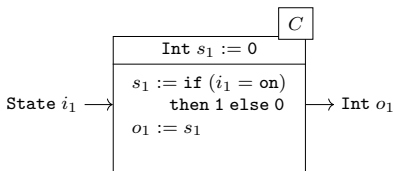
- Menge W : geschriebene Variablen in bereits übersetzten Zuweisungen
- $x := e$ wird zu $x' = w(e)$
- $w(e)$ ersetzt y durch y' in e für $y \in W$
- Γ : Konjunktion aller Zuweisungen

Syntax
abstrakt $\leftrightarrow$ konkret

Beispiel 1: Ausführung von Komponenten

Semantik

Konkrete Syntax:



Transitionen in $\rightarrow$:

- $\{s_1 \mapsto 0\} \xrightarrow{\{i_1 \mapsto \text{off}\} / \{o_1 \mapsto 0\}} \{s_1 \mapsto 0\}$
- $\{s_1 \mapsto 0\} \xrightarrow{\{i_1 \mapsto \text{on}\} / \{o_1 \mapsto 1\}} \{s_1 \mapsto 1\}$
- $\{s_1 \mapsto 1\} \xrightarrow{\{i_1 \mapsto \text{off}\} / \{o_1 \mapsto 0\}} \{s_1 \mapsto 0\}$
- $\{s_1 \mapsto 1\} \xrightarrow{\{i_1 \mapsto \text{on}\} / \{o_1 \mapsto 1\}} \{s_1 \mapsto 1\}$

Abstrakte Syntax:

$C = \langle \mathbf{i}, \mathbf{s}, \mathbf{o}, I, \Gamma \rangle$ mit

$\mathbf{i} = \{i_1\}$, $\mathbf{s} = \{s_1\}$, $\mathbf{o} = \{o_1\}$,

$I = (s_1 = 0)$ und

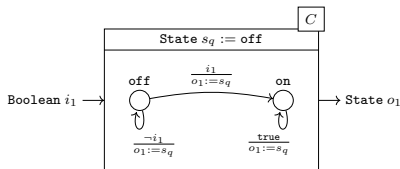
$\Gamma = (s'_1 = \text{if } (i_1 = \text{on}) \text{ then } 1 \text{ else } 0) \wedge (o'_1 = s'_1)$

Lauf (Anfang):

Var.	t_0	t_1	t_2	t_3	t_4
i_1	off	off	on	off	on
s_1	0	0	0	1	0
s'_1	0	0	1	0	1
o'_1	0	0	1	0	1

Zustandsbasierte Komponenten

Konkrete Syntax:



Abstrakte Syntax:

Syntax
abstrakt $\leftrightarrow$ konkret

$C = \langle \mathbf{i}, \mathbf{s}, \mathbf{o}, \mathbf{I}, \Gamma \rangle$ mit

$\mathbf{i} = \{i_1\}$, $\mathbf{s} = \{s_q\}$, $\mathbf{o} = \{o_1\}$,

$\mathbf{I} = (s_q = \text{off})$ und

$\Gamma = (s_q = \text{off} \wedge \neg i_1 \wedge s'_q = \text{off} \wedge o'_1 = s'_q) \vee$
 $(s_q = \text{off} \wedge i_1 \wedge s'_q = \text{on} \wedge o'_1 = s'_q) \vee$
 $(s_q = \text{on} \wedge \text{true} \wedge s'_q = \text{on} \wedge o'_1 = s'_q)$

Verhalten:

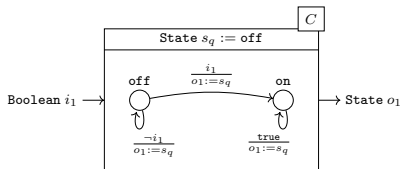
- Automat, spezielle Variable $s_q \in \mathbf{s}$ vom Typ State verwaltet Zustand
- Transitionen $\frac{g}{u}$ mit Bedingung $g \in \mathbb{F}_{\mathbf{s} \cup \mathbf{i}}$ und Zuweisungsliste u (wie bisher, s_q erste Zuweisung)

Übersetzung von Transitionen:

- $\frac{g}{u}$ wird zu $(g \wedge \varphi_u)$
- φ_u Übersetzung von u wie für kombinatorische Komponenten
- Γ : Disjunktion aller Transitionen

Beispiel 2: Ausführung von Komponenten

Konkrete Syntax:



Abstrakte Syntax:

Semantik

$C = \langle \mathbf{i}, \mathbf{s}, \mathbf{o}, I, \Gamma \rangle$ mit

$\mathbf{i} = \{i_1\}$, $\mathbf{s} = \{s_q\}$, $\mathbf{o} = \{o_1\}$,

$I = (s_q = \text{off})$ und

$\Gamma = (s_q = \text{off} \wedge \neg i_1 \wedge s'_q = \text{off} \wedge o'_1 = s'_q) \vee$
 $(s_q = \text{off} \wedge i_1 \wedge s'_q = \text{on} \wedge o'_1 = s'_q) \vee$
 $(s_q = \text{on} \wedge \text{true} \wedge s'_q = \text{on} \wedge o'_1 = s'_q)$

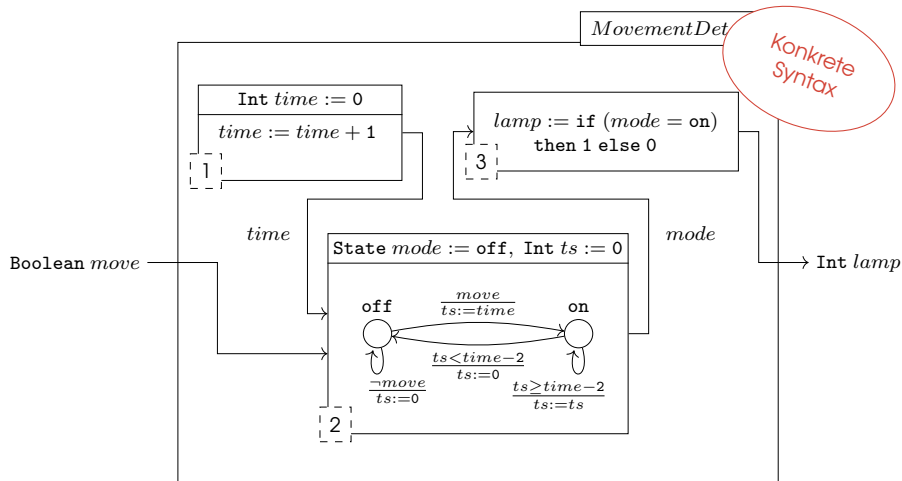
Transitionen in $\rightarrow$:

- $\{s_q \mapsto \text{off}\} \xrightarrow{\{i_1 \mapsto 0\} / \{o_1 \mapsto \text{off}\}} \{s_q \mapsto \text{off}\}$
- $\{s_q \mapsto \text{off}\} \xrightarrow{\{i_1 \mapsto 1\} / \{o_1 \mapsto \text{on}\}} \{s_q \mapsto \text{on}\}$
- $\{s_q \mapsto \text{on}\} \xrightarrow{\{i_1 \mapsto 0\} / \{o_1 \mapsto \text{on}\}} \{s_q \mapsto \text{on}\}$
- $\{s_q \mapsto \text{on}\} \xrightarrow{\{i_1 \mapsto 1\} / \{o_1 \mapsto \text{on}\}} \{s_q \mapsto \text{on}\}$

Lauf (Anfang):

Var.	t_0	t_1	t_2	t_3	t_4
i_1	0	0	1	1	0
s_q	off	off	off	on	on
s'_q	off	off	on	on	on
o'_1	off	off	on	on	on

Komposition von Komponenten (Beispiel)



Komposition von Komponenten (Operationen)

Umbenennung von Variablen:

- $C[x \mapsto y]$: Komponente, die durch Umbenennen der Variable x zu y in C entsteht

Abstrakte
Syntax

Verstecken von Variablen:

- $C \setminus x$: Komponente, die aus C durch Verstecken der Variable x entsteht (x wird aus $\mathbf{i}$ und/oder $\mathbf{o}$ entfernt und $\mathbf{s}$ hinzugefügt)

Sequenzielle Komposition:

- $C_1; C_2$: Komposition von $C_1 = \langle \mathbf{i}_1, \mathbf{s}_1, \mathbf{o}_1, I_1, \Gamma_1 \rangle$ und $C_2 = \langle \mathbf{i}_2, \mathbf{s}_2, \mathbf{o}_2, I_2, \Gamma_2 \rangle$

$C_1; C_2 = \langle \mathbf{i}, \mathbf{s}, \mathbf{o}, I, \Gamma \rangle$ mit

- $\mathbf{i} = \mathbf{i}_1 \cup \mathbf{i}_2$ und $\mathbf{s} = \mathbf{s}_1 \cup \mathbf{s}_2$ und $\mathbf{o} = \mathbf{o}_1 \cup \mathbf{o}_2$,
- $I = I_1 \wedge I_2$, und
- $\Gamma = \Gamma_1 \wedge w(\Gamma_2)$ $w(e)$ ersetzt y durch y' in e für $y \in (\mathbf{s}_1 \cup \mathbf{o}_1)$

Bedingung: $(\mathbf{i}_1 \cup \mathbf{s}_1 \cup \mathbf{o}_1) \cap (\mathbf{s}_2 \cup \mathbf{o}_2) = \emptyset$ und $\mathbf{i}_2 \cap (\mathbf{s}_1 \setminus \mathbf{o}_1) = \emptyset$

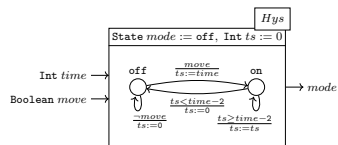
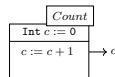
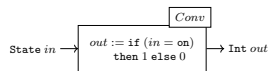
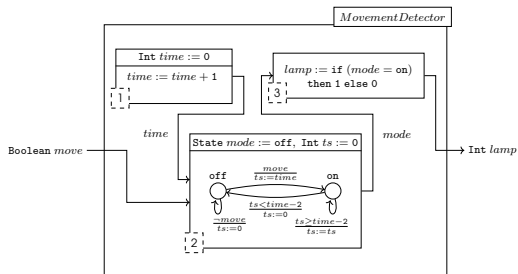
Beispiel: Komposition

Syntax
abstrakt $\leftrightarrow$ konkret

MovementDetector =

$(\text{Count}[c \mapsto \text{time}]; \text{Hys};$

$\text{Conv}[\text{in} \mapsto \text{mode}][\text{out} \mapsto \text{lamp}]) \setminus \text{time} \setminus \text{mode}$



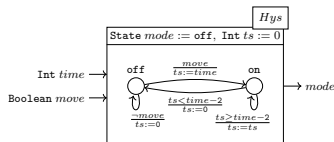
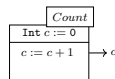
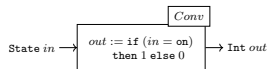
Beispiel Komposition (contd.)

Syntax
abstrakt $\leftrightarrow$ konkret

MovementDetector =
 $\left(\text{Count}[c \mapsto \text{time}]; \text{Hys}; \right.$
 $\left. \text{Conv}[in \mapsto \text{mode}][out \mapsto \text{lamp}] \right) \setminus \text{time} \setminus \text{mode}$

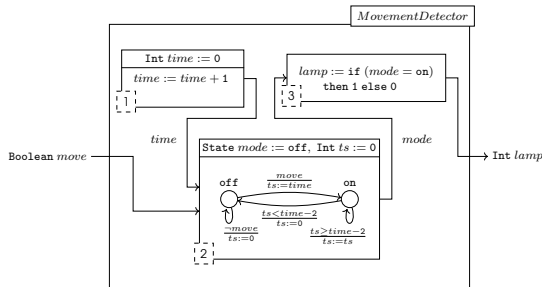
MovementDetector = $\langle i, s, o, I, \Gamma \rangle$ mit:

- $i = \{\text{move}\}, s = \{\text{time}, \text{mode}, \text{ts}\}, o = \{\text{lamp}\}$
- $I = (\text{time} = 0 \wedge \text{mode} = \text{off} \wedge \text{ts} = \text{ts})$
- $\Gamma = (\text{time}' = \text{time} + 1) \wedge$
 $\left((\text{mode} = \text{off} \wedge \neg \text{move} \wedge \text{mode}' = \text{off} \wedge \text{ts} = 0) \vee \right.$
 $(\text{mode} = \text{off} \wedge \text{move} \wedge \text{mode}' = \text{on} \wedge \text{ts}' = \text{time}') \vee$
 $(\text{mode} = \text{on} \wedge \text{ts} \geq \text{time}' - 2 \wedge \text{mode}' = \text{on} \wedge \text{ts}' = \text{ts}) \vee$
 $(\text{mode} = \text{on} \wedge \text{ts} < \text{time}' - 2 \wedge \text{mode}' = \text{off} \wedge \text{ts}' = 0) \Big) \wedge$
 $(\text{lamp}' = \text{if } (\text{mode}' = \text{on}) \text{ then } 1 \text{ else } 0)$



Beispiel Komposition (contd.)

Semantik



$MovementDetector = \langle i, s, o, I, \Gamma \rangle$ mit:

- $i = \{move\}, s = \{time, mode, ts\}, o = \{lamp\}$
- $I = (time = 0 \wedge mode = off \wedge ts = ts)$
- $\Gamma = (time' = time + 1) \wedge$
 $((mode = off \wedge \neg move \wedge mode' = off \wedge ts = 0) \vee$
 $(mode = off \wedge move \wedge mode' = on \wedge ts' = time') \vee$
 $(mode = on \wedge ts \geq time' - 2 \wedge mode' = on \wedge ts' = ts) \vee$
 $(mode = on \wedge ts < time' - 2 \wedge mode' = off \wedge ts' = 0)) \wedge$
 $(lamp' = if (mode' = on) then 1 else 0)$

Lauf (Anfang):

Var.	t_0	t_1	t_2	t_3	t_4
<i>move</i>	0	1	0	0	1
<i>time</i>	0	1	2	3	4
<i>mode</i>	off	off	on	on	on
<i>ts</i>	0	0	2	2	2
<i>time'</i>	1	2	3	4	5
<i>mode'</i>	off	on	on	on	off
<i>ts'</i>	0	2	2	2	0
<i>lamp'</i>	0	1	1	1	0

Diskussion: Komponenten und Komposition

Syntax vs. Semantik:

- Komposition: Syntaktisches Werkzeug
- Unterliegende Semantik für alle Komponenten gleich

Top-Down / Bottom-up:

- Architektur und Struktur: Dekomposition (Top-Down)
- Semantik / Verhalten: Komposition (Bottom-Up)

Vereinfachtes Modell:

- Feste Reihenfolgenfolge in Komponente
- Nur Datenfluss, kein Kontrollfluss
- Keine Ereignisse / Nachrichten / komplexe Datentypen

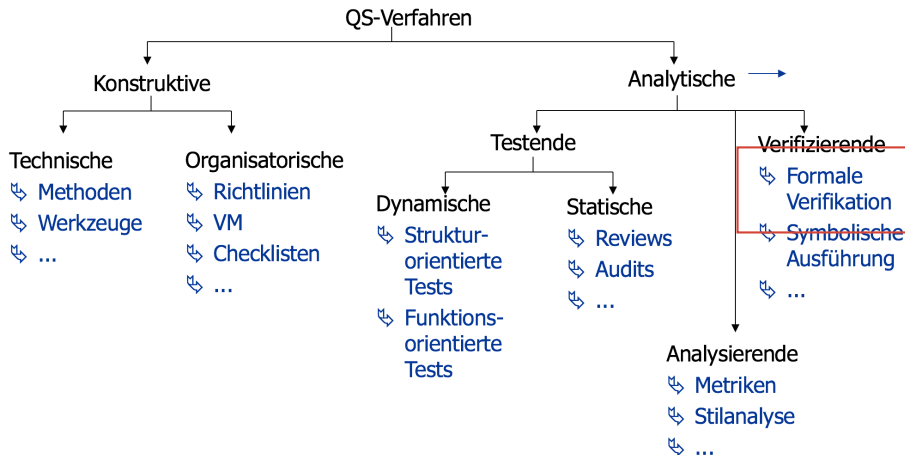
In der Praxis:

- Modellbasierte Softwareentwicklung in der Automobilbranche und in der Luftfahrt
- Regelungstechnische Systeme

Agenda

1. Motivation und Begriffe
2. (Code) Standards
3. (Code) Reviews
4. Modellierung und Generierung
5. Verifikation: Model-Checking und Invarianten
6. Metriken und Statische Code Analyse
7. Formale Semantik and Symbolische Ausführung
8. Literatur und Standards

QS - Klassifizierung von Ansätzen



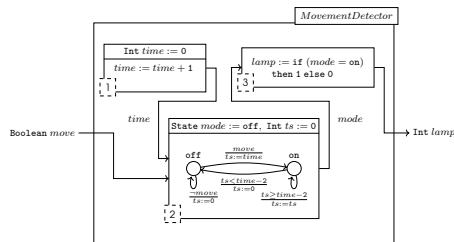
Analyse des Verhaltens von Komponenten

Semantische Korrektheit:

- Erfüllt das Modell funktionale Anforderungen?

In diesem Abschnitt:

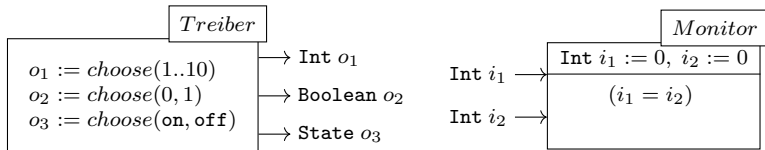
- Analyse von Modellen
- Bounded Model Checking
- Induktive Invarianten



Beispiel:

- Frage: Haben wir korrekt modelliert?
- Z.B. Geht das Licht an, wenn sich etwas bewegt?

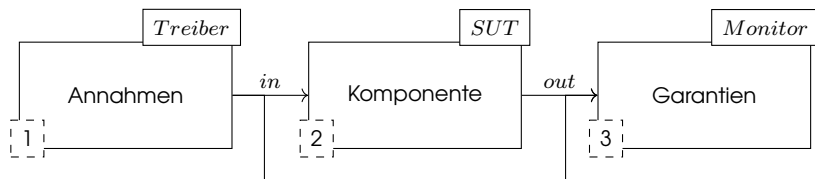
Testen mit Treiber und Monitor



$$(o'_1 \geq 1 \wedge o'_1 \leq 10) \wedge (o'_2 \vee \neg o'_2) \wedge (o'_3 = \text{on} \vee o'_3 = \text{off})$$

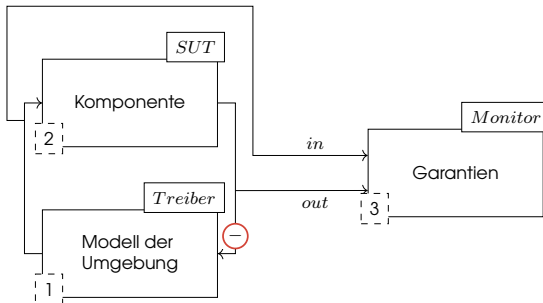
- **Treiber** liefern Werte für alle Eingabevariablen der zu testenden Komponente (Annahmen über Wahl der Werte)
- **Monitore** beschreiben zu prüfende Eigenschaften und machen Ausgaben der Komponente zu Zustand

Open-Loop Analyse



- SUT = System unter Test (zu testende Komponente)
- Open-Loop: Treiber und Komponente in Reihe, kein Feedback
- Annahmen über Umgebung nur durch Eingaben
- Test-Aufbau in dieser Veranstaltung

Closed-Loop Analyse



- SUT = System under Test (zu testende Komponente)
- Closed-Loop: Treiber und Komponente in Feedback-Schleife
- Annahmen über Umgebung durch Feedback (z.B. Regelungstechnische Systeme)
- **Achtung:** Keine Eingaben für Treiber bei erstem Rechnen

Transitionssysteme

- Kombination von Komponente, Treiber und Monitor hat keine Eingaben und Ausgaben
- Ohne Eingaben und Ausgaben können wir unsere Definition vereinfachen:

Definition (Transitionssystem)

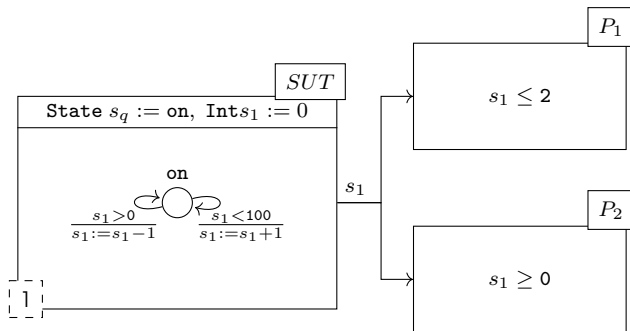
Ein Transitionssystem ist ein Tupel $\langle s, I, \Gamma \rangle$ mit

- einer Menge s von getypten Variablen, die eine Menge von Zuständen V_s (mögliche Belegungen von s) aufspannen,
- einem Booleschen Ausdruck $I \in \mathbb{F}_s$ über s der die initialen Zustände $\llbracket I \rrbracket \subseteq V_s$ beschreibt und
- einem Booleschen Ausdruck $\Gamma \in \mathbb{F}_{s \cup s'}$ der die Transitionsrelation $\llbracket \Gamma \rrbracket \subseteq V_s \times V_s$ beschreibt.

Transitionen:

- $(v_s, v'_s) \in \llbracket \Gamma \rrbracket$ wenn $v_s \circ v'_s[s'/s] \models \Gamma$

Beispiel



- SUT nicht-deterministisch; erhöht oder verringert s_1 um 1 in Grenzen 0 bis 100
- Zwei Monitore für P_1 (gilt nicht immer) und P_2 (gilt immer)

Beispiel (contd.)

Sei $\langle s, I, \Gamma \rangle$ mit $s = \{s_q, s_1\}$ mit s_q vom Typ **State** und s_1 vom Typ **Int**.

Initialisierung:

Für $I = (s_q = \text{on} \wedge s_1 = 0)$ ist $\llbracket I \rrbracket = \{s_q \mapsto \text{on}, s_1 \mapsto 0\}$.

Transitionen:

Für $\Gamma = ((s_q = \text{on}) \wedge (s_1 < 100) \wedge (s'_q = \text{on}) \wedge (s'_1 = s_1 + 1)) \vee$
 $((s_q = \text{on}) \wedge (s_1 > 0) \wedge (s'_q = \text{on}) \wedge (s'_1 = s_1 - 1))$

Dann ist $\llbracket \Gamma \rrbracket = \{ (\{s_q \mapsto \text{on}, s_1 \mapsto 0\}, \{s_q \mapsto \text{on}, s_1 \mapsto 1\}),$
 $\dots,$
 $(\{s_q \mapsto \text{on}, s_1 \mapsto 1\}, \{s_q \mapsto \text{on}, s_1 \mapsto 0\}),$
 $(\{s_q \mapsto \text{on}, s_1 \mapsto -2\}, \{s_q \mapsto \text{on}, s_1 \mapsto -1\}),$
 $\dots \}$

Erreichbare Belegungen

Definition (Erreichbare Belegungen)

In $T = \langle s, I, \Gamma \rangle$ ist eine Belegung $v_s \in V_s$ erreichbar wenn es eine Sequenz von Belegungen $v_0, v_1, \dots, v_k$ in V_s gibt, so dass

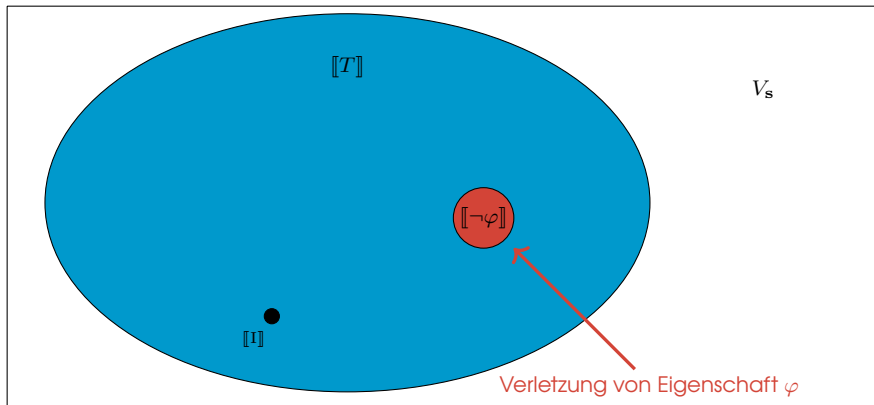
$$v_0 \in \llbracket I \rrbracket \quad \text{und} \quad (v_{i-1}, v_i) \in \llbracket \Gamma \rrbracket \quad \text{für} \quad 1 \leq i \leq k \quad \text{und} \quad v_k = v_s.$$

Wir schreiben $\llbracket T \rrbracket$ für die Menge aller erreichbaren Belegungen von T .

Beispiel:

- Sei $T = \langle s, I, \Gamma \rangle$ wie auf der vorigen Folie
- Dann ist $(\{s_q \mapsto \text{on}, s_1 \mapsto -2\}, \{s_q \mapsto \text{on}, s_1 \mapsto -1\}) \in \llbracket \Gamma \rrbracket$
- Aber $\llbracket T \rrbracket = \{ \{s_q \mapsto \text{on}, s_1 \mapsto 0\}, \{s_q \mapsto \text{on}, s_1 \mapsto 1\}, \dots, \{s_q \mapsto \text{on}, s_1 \mapsto 100\} \}$

Zustandsraum und Eigenschaften



- V_s und $\llbracket T \rrbracket$ generell nicht endlich

Mögliche Ergebnisse einer Analyse

Frage:

Gilt Eigenschaft P für Transitionssystem T bzw.
gilt P allen erreichbaren Belegungen in $\llbracket T \rrbracket$?

Mögliche Antworten mit Fehlern:

- **True Positive (TP)** Antwort: P kann verletzt werden. Und P wird tatsächlich verletzt.
Ein Fehler liegt vor und wird gefunden
- **True Negative (TN)** Antwort: Keine Verletzung von P gefunden. Und P wird
tatsächlich nicht verletzt. Es liegt kein Fehler vor und es wird auch keiner berichtet
- **False Positive (FP)** Antwort: P kann verletzt werden. Aber: P wird tatsächlich nicht
verletzt. Es wird ein Fehler berichtet, obwohl kein Fehler vorliegt
- **False Negative (FN)** Antwort: Keine Verletzung von P gefunden. Aber P wird verletzt.
Es wird kein Fehler berichtet, obwohl ein Fehler vorliegt

Bounded Model Checking (BMC)



- Model Checking = Systematisches Explorieren der Zustände eines Systems zur Überprüfung von Eigenschaften
- Unendlich viele Zustände $\Rightarrow$ Vollständige Exploration evtl. unmöglich

k -Sicherheit

- Für $T = \langle s, I, \Gamma \rangle$ sei $\Gamma^{i,j} = \Gamma[s^i/s][s^j/s']$
- Für Eigenschaft P sei $P^k = P[s^k/s]$

Definition (k -Sicherheit)

Ein Transitionssystem $T = \langle s, I, \Gamma \rangle$ ist k -sicher für eine Eigenschaft P , wenn P für min. k Schritte gilt:

$$(I[s^0/s] \wedge \Gamma^{0,1} \wedge \dots \wedge \Gamma^{k-1,k}) \models (P^0 \wedge \dots \wedge P^k).$$

- 0-Sicherheit: $I[s^0/s] \models P^0$
- 1-Sicherheit: $(I[s^0/s] \wedge \Gamma^{0,1}) \models (P^0 \wedge P^1)$
- ...

Wir schreiben $a \models b$ für a impliziert b

Bounded Model Checking

Wdh. Implikation:

- Zur Erinnerung: Wir testen $(a \models b)$, indem wir nach einem Modell für $(a \wedge \neg b)$ suchen (vgl. Folien SWK-2 Anforderungen und Spezifikation).
- Wenn es kein Modell für $(a \wedge \neg b)$ gibt, dann gilt $(a \models b)$

Test für k -Sicherheit:

- Wir testen k -Sicherheit von $T = \langle s, I, \Gamma \rangle$ bzgl. P indem wir nach einem Modell suchen für

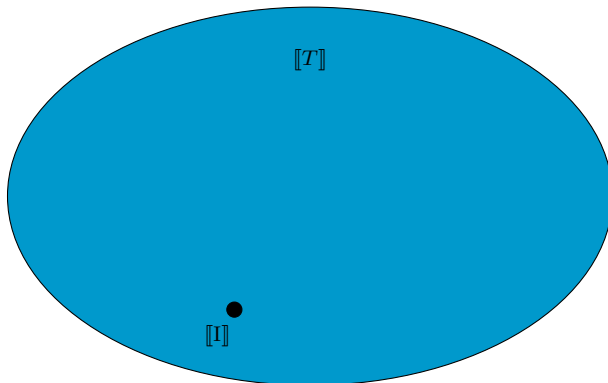
$$I[s^0/s] \wedge \Gamma^{0,1} \wedge \dots \wedge \Gamma^{k-1,k} \wedge (\neg P^0 \vee \dots \vee \neg P^k).$$

Definition (Bounded Model Checking)

Für Transitionssystem $T = \langle s, I, \Gamma \rangle$ und $k = 0$:

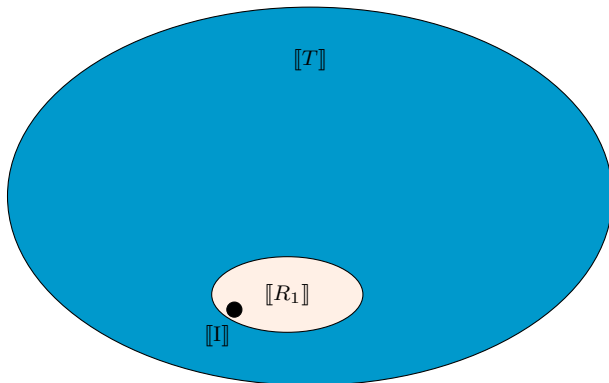
- ① Wenn Test für k -Sicherheit von T erfüllbar ist,
- ② dann Fehler gefunden! Stop mit Modell als Gegenbeispiel.
- ③ Sonst $k = k + 1$.
- ④ Gehe zu (1).

Intuition: Bounded Model Checking



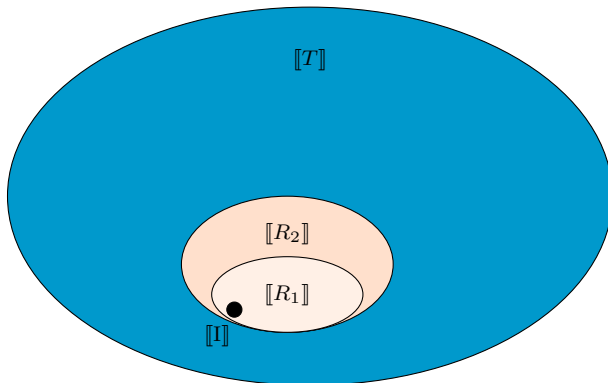
- $\llbracket R_i \rrbracket$: Menge der Belegungen, die in max. i Schritten erreicht werden.
- Es gilt $\llbracket R_i \rrbracket \subseteq \llbracket R_{i+1} \rrbracket$
- Es gibt keine Möglichkeit festzustellen, wann $\llbracket T \rrbracket \subseteq \llbracket R_i \rrbracket$ da wir keine logische Formel haben, die $\llbracket T \rrbracket$ beschreibt

Intuition: Bounded Model Checking



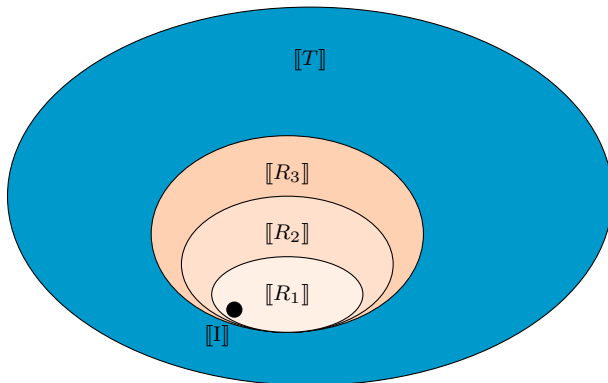
- $\llbracket R_i \rrbracket$: Menge der Belegungen, die in max. i Schritten erreicht werden.
- Es gilt $\llbracket R_i \rrbracket \subseteq \llbracket R_{i+1} \rrbracket$
- Es gibt keine Möglichkeit festzustellen, wann $\llbracket T \rrbracket \subseteq \llbracket R_i \rrbracket$ da wir keine logische Formel haben, die $\llbracket T \rrbracket$ beschreibt

Intuition: Bounded Model Checking



- $\llbracket R_i \rrbracket$: Menge der Belegungen, die in max. i Schritten erreicht werden.
- Es gilt $\llbracket R_i \rrbracket \subseteq \llbracket R_{i+1} \rrbracket$
- Es gibt keine Möglichkeit festzustellen, wann $\llbracket T \rrbracket \subseteq \llbracket R_i \rrbracket$ da wir keine logische Formel haben, die $\llbracket T \rrbracket$ beschreibt

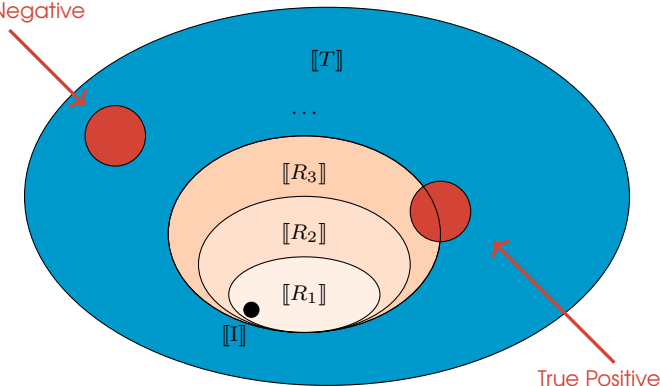
Intuition: Bounded Model Checking



- $\llbracket R_i \rrbracket$: Menge der Belegungen, die in max. i Schritten erreicht werden.
- Es gilt $\llbracket R_i \rrbracket \subseteq \llbracket R_{i+1} \rrbracket$
- Es gibt keine Möglichkeit festzustellen, wann $\llbracket T \rrbracket \subseteq \llbracket R_i \rrbracket$ da wir keine logische Formel haben, die $\llbracket T \rrbracket$ beschreibt

Intuition: Bounded Model Checking

False Negative



True Positive

- $\llbracket R_i \rrbracket$: Menge der Belegungen, die in max. i Schritten erreicht werden.
- Es gilt $\llbracket R_i \rrbracket \subseteq \llbracket R_{i+1} \rrbracket$
- Es gibt keine Möglichkeit festzustellen, wann $\llbracket T \rrbracket \subseteq \llbracket R_i \rrbracket$ da wir keine logische Formel haben, die $\llbracket T \rrbracket$ beschreibt

Beispiel: Bounded Model Checking

Sei $\langle s, I, \Gamma \rangle$ mit $s = \{s_q, s_1\}$ wie bisher und $I = (s_q = \text{on} \wedge s_1 = 0)$

Transitionen:

$$\begin{aligned} \text{Für } \Gamma = & ((s_q = \text{on}) \wedge (s_1 < 100) \wedge (s'_q = \text{on}) \wedge (s'_1 = s_1 + 1)) \vee \\ & ((s_q = \text{on}) \wedge (s_1 > 0) \wedge (s'_q = \text{on}) \wedge (s'_1 = s_1 - 1)) \end{aligned}$$

$$\begin{aligned} \text{ist } \Gamma^{0,1} = & ((s_q^0 = \text{on}) \wedge (s_1^0 < 100) \wedge (s_q^1 = \text{on}) \wedge (s_1^1 = s_1^0 + 1)) \vee \\ & ((s_q^0 = \text{on}) \wedge (s_1^0 > 0) \wedge (s_q^1 = \text{on}) \wedge (s_1^1 = s_1^0 - 1)) \end{aligned}$$

Test für 1-Sicherheit bzgl. P_1 ($s_1 \leq 2$):

$$\begin{aligned} & (s_q^0 = \text{on} \wedge s_1^0 = 0) \wedge && // I[s^0/s] \\ & (((s_q^0 = \text{on}) \wedge (s_1^0 < 100) \wedge (s_q^1 = \text{on}) \wedge (s_1^1 = s_1^0 + 1)) \vee && // \Gamma^{0,1} \\ & ((s_q^0 = \text{on}) \wedge (s_1^0 > 0) \wedge (s_q^1 = \text{on}) \wedge (s_1^1 = s_1^0 - 1))) \wedge \\ & (\neg(s_1^0 \leq 2) \vee \neg(s_1^1 \leq 2)) && // \neg P^0 \vee \neg P^1 \end{aligned}$$

T ist 2-sicher aber nicht 3-sicher für P_1 !

Z3 Beispiel: <https://rise4fun.com/Z3/IC5D>

Invarianten



- Manchmal wollen wir **beweisen**, dass eine Eigenschaft immer gilt
- Ein Invariante ist eine Eigenschaft, die in jedem Zustand eines Systems gilt
- Typischerweise über-approximiert eine Invariante $\llbracket T \rrbracket$ ist dafür aber einfach logisch ausdrückbar

Induktive Invarianten

Definition (Induktive Invariante)

Eine Eigenschaft P ist eine induktive Invariante von $T = \langle s, I, \Gamma \rangle$ wenn gilt dass:

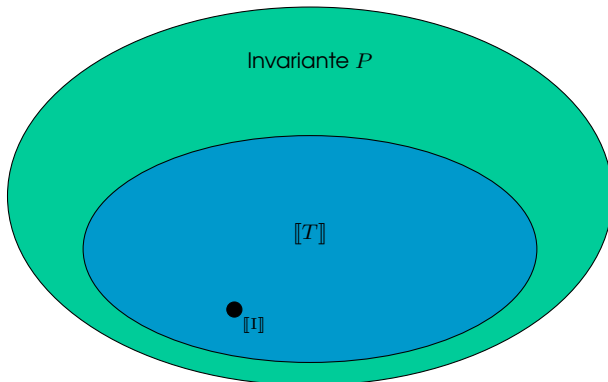
- ① jede Belegung in $v_s \in \llbracket I \rrbracket$ die Eigenschaft P erfüllt ($v_s \models P$), und
- ② falls eine Belegung v_s die Eigenschaft P erfüllt und (v_s, v'_s) in $\llbracket \Gamma \rrbracket$ ist, dann erfüllt auch v'_s die Eigenschaft P .

Induktionsbeweis:

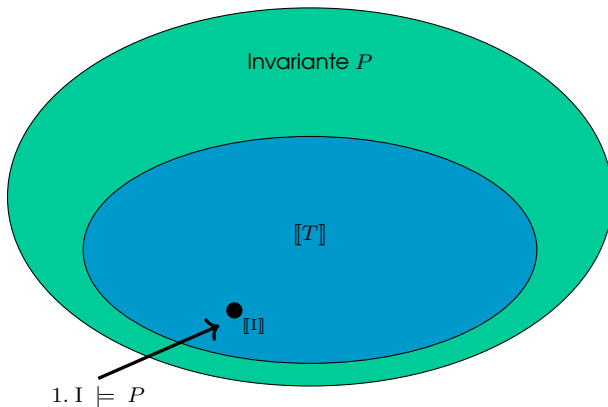
1. Basis: $I \models P$

2. Schritt: $P \wedge \Gamma \models P[s'/s]$

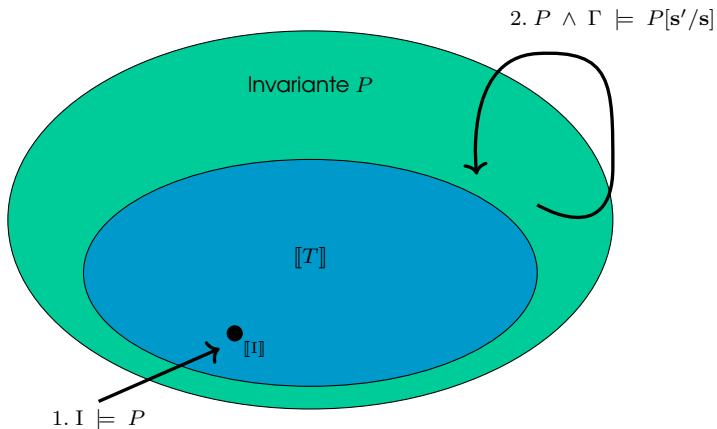
Intuition: Induktive Invarianten



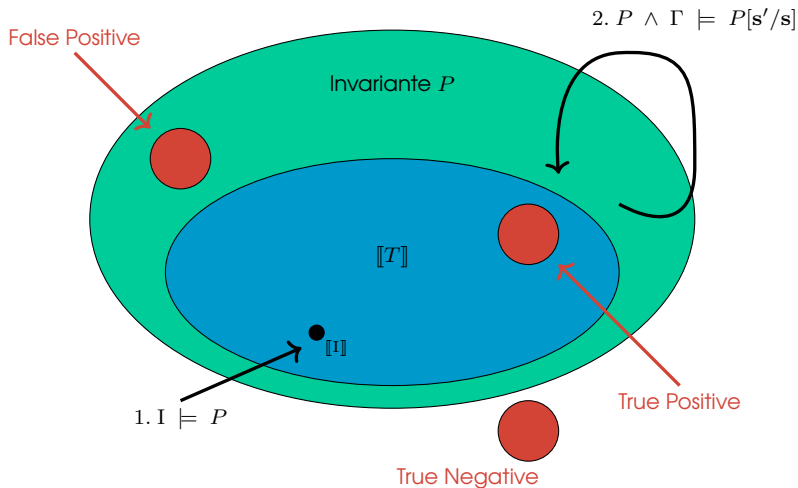
Intuition: Induktive Invarianten



Intuition: Induktive Invarianten



Intuition: Induktive Invarianten



Beispiel: Induktive Invariante

Sei $\langle s, I, \Gamma \rangle$ mit $s = \{s_q, s_1\}$ wie bisher mit $I = (s_q = \text{on} \wedge s_1 = 0)$ und

$$\Gamma = ((s_q = \text{on}) \wedge (s_1 < 100) \wedge (s'_q = \text{on}) \wedge (s'_1 = s_1 + 1)) \vee \\ ((s_q = \text{on}) \wedge (s_1 > 0) \wedge (s'_q = \text{on}) \wedge (s'_1 = s_1 - 1))$$

Wir beweisen $P_2 (s_1 \geq 0)$:

Beispiel: Induktive Invariante

Sei $\langle s, I, \Gamma \rangle$ mit $s = \{s_q, s_1\}$ wie bisher mit $I = (s_q = \text{on} \wedge s_1 = 0)$ und

$$\Gamma = ((s_q = \text{on}) \wedge (s_1 < 100) \wedge (s'_q = \text{on}) \wedge (s'_1 = s_1 + 1)) \vee \\ ((s_q = \text{on}) \wedge (s_1 > 0) \wedge (s'_q = \text{on}) \wedge (s'_1 = s_1 - 1))$$

Wir beweisen $P_2 (s_1 \geq 0)$:

1. Basis: $I \models P$

$$(s_q = \text{on} \wedge s_1 = 0) \models (s_1 \geq 0) \quad \checkmark$$

Beispiel: Induktive Invariante

Sei $\langle s, I, \Gamma \rangle$ mit $s = \{s_q, s_1\}$ wie bisher mit $I = (s_q = \text{on} \wedge s_1 = 0)$ und

$$\Gamma = ((s_q = \text{on}) \wedge (s_1 < 100) \wedge (s'_q = \text{on}) \wedge (s'_1 = s_1 + 1)) \vee \\ ((s_q = \text{on}) \wedge (s_1 > 0) \wedge (s'_q = \text{on}) \wedge (s'_1 = s_1 - 1))$$

Wir beweisen $P_2 (s_1 \geq 0)$:

1. Basis: $I \models P$

$$(s_q = \text{on} \wedge s_1 = 0) \models (s_1 \geq 0) \quad \checkmark$$

2. Schritt: $P \wedge \Gamma \models P[s'/s]$

$$(s_1 \geq 0) \wedge (((s_q = \text{on}) \wedge (s_1 < 100) \wedge (s'_q = \text{on}) \wedge (s'_1 = s_1 + 1)) \vee \\ ((s_q = \text{on}) \wedge (s_1 > 0) \wedge (s'_q = \text{on}) \wedge (s'_1 = s_1 - 1))) \models (s'_1 \geq 0) \quad \checkmark$$

Beispiel: Induktive Invariante (SMT)

Sei $\langle s, I, \Gamma \rangle$ und P wie eben

1. Basis: $I \models P$

$$(s_q = \text{on} \wedge s_1 = 0) \wedge \neg(s_1 \geq 0) \quad \text{unsat}$$

2. Schritt: $P \wedge \Gamma \models P[s'/s]$

$$(s_1 \geq 0) \wedge \left(((s_q = \text{on}) \wedge (s_1 < 100) \wedge (s'_q = \text{on}) \wedge (s'_1 = s_1 + 1)) \vee \right. \\ \left. ((s_q = \text{on}) \wedge (s_1 > 0) \wedge (s'_q = \text{on}) \wedge (s'_1 = s_1 - 1)) \right) \wedge \neg(s'_1 \geq 0) \quad \text{unsat}$$

Wenn es kein Modell für $(a \wedge \neg b)$ gibt, dann gilt $(a \models b)$

Z3 Beispiel: <https://rise4fun.com/Z3/0LXA4>

Diskussion: BMC und Invarianten

Bounded Model Checking

- Iteratives Abrollen des Transitionssystems
- Keine Möglichkeit festzustellen, wann/ob alle erreichbaren Belegungen untersucht

Fehler:

- False Positives: Nein, Modell für BMC Formel ist direkt ein Zeuge für Verletzung der Eigenschaft
- False Negatives: Möglich, u.U. nicht genug Zeit / Ressourcen um Transitionssystem tief genug abzurollen

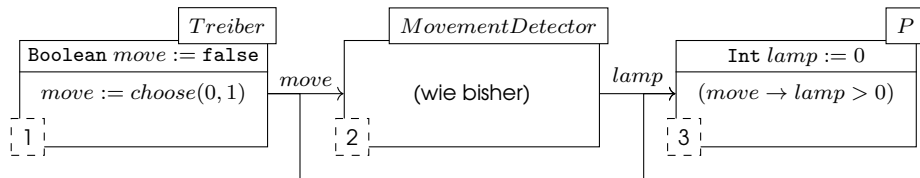
Induktive Invarianten

- Induktionsbeweis
- Über-approximiert $\llbracket T \rrbracket$

Fehler:

- False Positives: Möglich, durch Über-Approximierung
- False Negatives: Nein, da Beweis

Beispiel: Remote Home Control

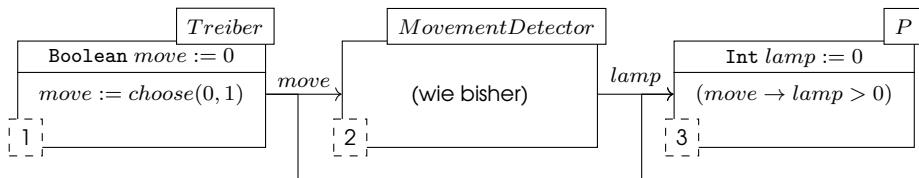


Wir missbrauchen die Notation etwas: eigentlich müsste die Initialisierung von *lamp* zu *MovementDetector* gehören.

Transitionssystem $Treiber; MovementDetector; P = \langle s, I, \Gamma \rangle$ mit:

- $s = \{move, time, mode, ts, lamp\}$
- $I = (\neg move \wedge time = 0 \wedge mode = off \wedge ts = 0 \wedge lamp = 0)$
- $\Gamma = (move' \vee \neg move') \wedge (time' = time + 1) \wedge$
 $\left((mode = off \wedge \neg move' \wedge mode' = off \wedge ts = 0) \vee \right.$
 $(mode = off \wedge move' \wedge mode' = on \wedge ts' = time') \vee$
 $(mode = on \wedge ts \geq time' - 2 \wedge mode' = on \wedge ts' = ts) \vee$
 $\left. (mode = on \wedge ts < time' - 2 \wedge mode' = off \wedge ts' = 0) \right) \wedge$
 $(lamp' = \text{if } (mode' = on) \text{ then } 1 \text{ else } 0)$

Beispiel: Remote Home Control



Wir missbrauchen die Notation etwas: eigentlich müsste die Initialisierung von `lamp` zu `MovementDetector` gehören.

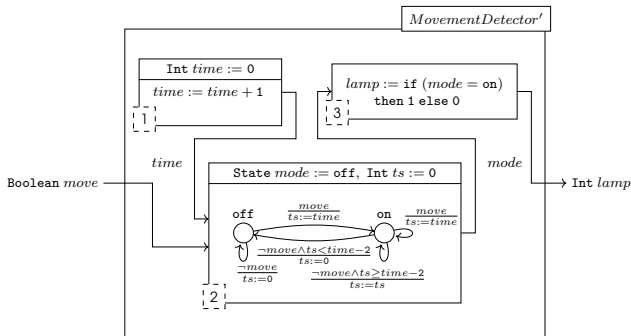
- **Gewünschte Eigenschaft:**

Wenn Bewegung dann Licht an ($move \rightarrow lamp > 0$)

- **BMC:**

Eigenschaft ist nicht 3-sicher (Licht geht nach Nachlauf erst einmal aus)!

Beispiel: Remote Home Control



■ Anpassung:

MovementDetector': Extra Transition im Automaten für Fall `mode = on` und `move`

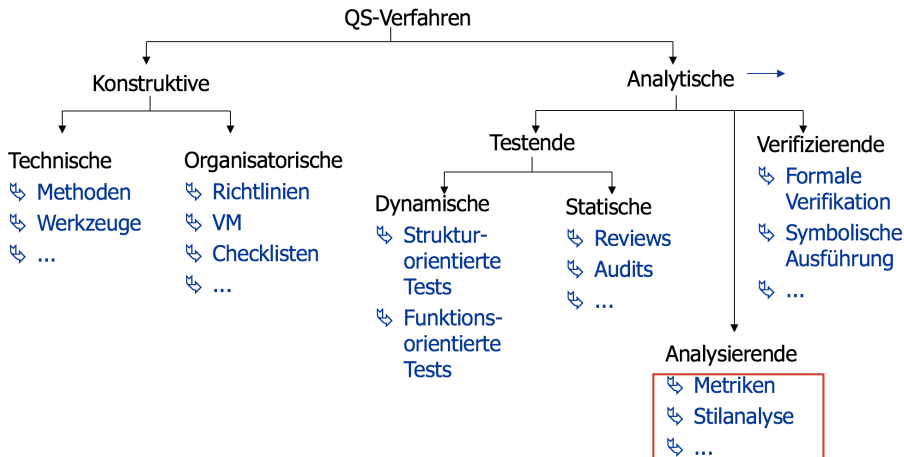
■ Invariante:

$(move \rightarrow lamp > 0)$ beweisbar auf *MovementDetector'*

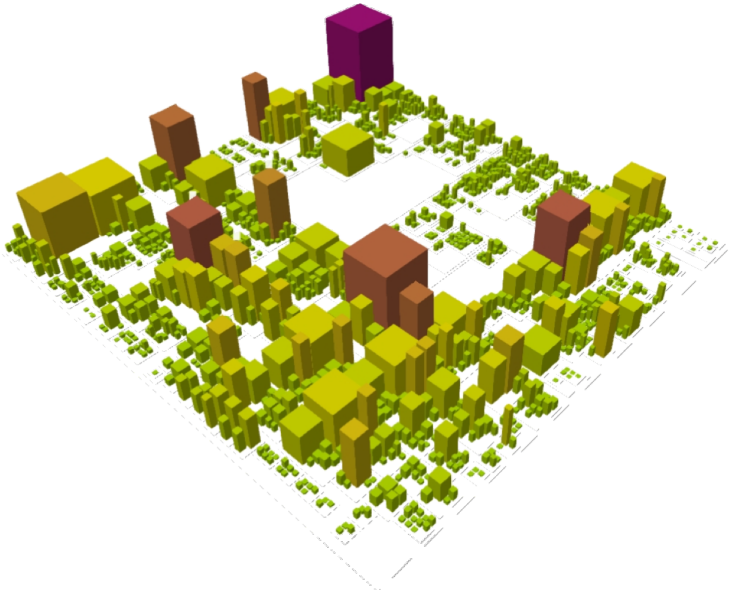
Agenda

1. Motivation und Begriffe
2. (Code) Standards
3. (Code) Reviews
4. Modellierung und Generierung
5. Verifikation: Model-Checking und Invarianten
- 6. Metriken und Statische Code Analyse**
7. Formale Semantik and Symbolische Ausführung
8. Literatur und Standards

QS - Klassifizierung von Ansätzen



Code Metriken



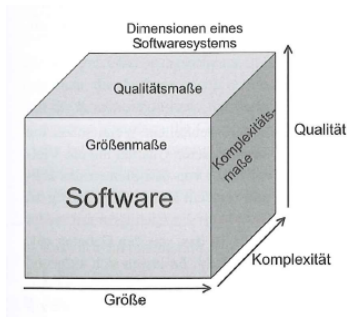
Code Metriken

Definition (Code Metrik)

Eine Code Metrik ist eine Maßeinheit und eine Methode zur Berechnung eines Meßwertes, welcher den Grad ausdrückt, zu welchem Software eine Eigenschaft erfüllt.

Beispiele:

- LOC: Lines of Code
- LCOM: Lack of Cohesion
- CC: Cyclomatic Complexity



Lines of Code (LOC)

Definition (Lines of Code (LOC))

LOC ist die Anzahl der Zeilen im Code einer Software.

- Einfachster Indikator für Komplexität einer Software.
- Ähnliche Metriken: Number of Classes, Number of Methods, Number of ...

Lack of Cohesion Metric (LCOM)

Kohäsion: Innerer Zusammenhang (vgl. Vorlesung Architektur)

Lack of Cohesion Metric (LCOM)

Für eine Klasse mit a Feldern und m Methoden sei $n(A_j)$ die Zahl der Methoden, die auf Feld A_j zugreifen. LCOM ist dann

$$\frac{\left(\frac{1}{a} \sum_{i=1}^a n(A_i)\right) - m}{1 - m}.$$

- Zusammenhang wird geschätzt durch Zugriff
- Hoher LCOM Wert $\Leftrightarrow$ geringe Kohäsion

Beispiel: Lack of Cohesion Metric (LCOM)

Dreieck
-a : int
-b : int
-c : int
+setKannten(a:int, b:int, c:int)
+zeichne()
+umfang()
+flaeche()

Kreis
-r : int
+setRadius(r:int)
+zeichne()
+umfang()
+flaeche()

Dreieck and Kreis:
Jedes Feld wird von jeder Methode genutzt.

Dreieck:

$$a = 3, m = 4, A_1 = 4, A_2 = 4, A_3 = 4$$
$$\Rightarrow LCOM = \frac{(1/3 \cdot 12) - 4}{1 - 4} = 0$$

Kreis:

$$a = 1, m = 4, A_1 = 4$$
$$\Rightarrow LCOM = \frac{(1/1 \cdot 4) - 4}{1 - 4} = 0$$

Figur
-a : int
-b : int
-c : int
-r : int
+zeichneDreieck()
+zeichneKreis()
+flaecheDreieck()
+flaecheKreis()
+umfangDreieck()
+umfangKreis()
+setRadius(r:int)
+setKannten(a:int, b:int, c:int)

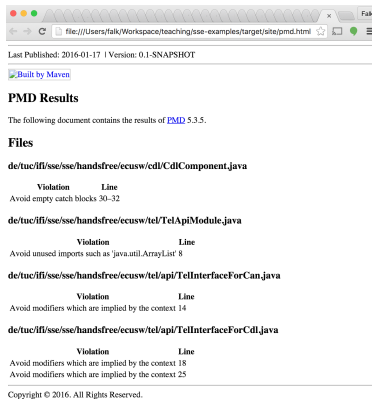
Figur = Dreieck + Kreis:

$$a = 4, m = 8, A_1 = 4, A_2 = 4, A_3 = 4, A_4 = 4$$
$$\Rightarrow LCOM = \frac{(1/4 \cdot 16) - 8}{1 - 8} = \frac{4}{7}$$

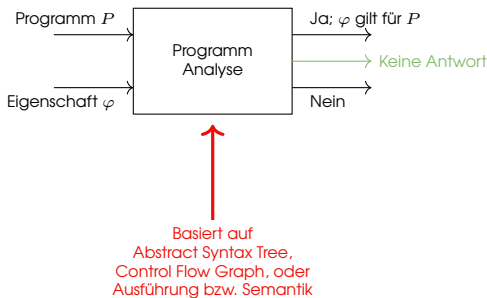
Beispiel: Werkzeug

PMD sucht in Java Code nach:

- Doppeltem Code
- Komplizierten Ausdrücken
- Vielen anderen Eigenschaften



Programmanalyse



Beispiele:

- Alle **Klassennamen** beginnen mit Großbuchstaben
- Alle **Variablen** werden initialisiert
- **Anweisung** L10 kann nicht erreicht werden

Statische Analyse

- Analyse beim Kompilieren
- Basiert auf Modell, AST oder CFG
- Prüft Eigenschaften ohne Code auszuführen
- Nutzt oft **Über-Approximierung** von möglichem Verhalten des Codes (kein Beispiel in dieser Vorlesung!)

Wdh. Getypte Ausdrücke

$a \in \mathbf{AExp}$ (Arithmetische Ausdrücke)

$a ::= x \mid \mathbf{n} \mid (a_1 \text{ } op_a \text{ } a_2)$

$b \in \mathbf{BExp}$ (Boolesche Ausdrücke)

$b ::= y \mid \mathbf{true} \mid \mathbf{false} \mid \neg b \mid (b_1 \text{ } op_b \text{ } b_2) \mid (a_1 \text{ } op_r \text{ } a_2)$

x Variable Typ $\mathbf{Int}$

y Variable Typ $\mathbf{Boolean}$

$\mathbf{n}$ Literal für $n \in \mathbb{Z}$

$op_a \in \mathbf{Op_a}$ arithmetische Op.: $+$, $-$

$op_b \in \mathbf{Op_b}$ Boolesche Op.: $\wedge$, $\vee$

$op_r \in \mathbf{Op_r}$ Relationale Op.: $>$, $<$, $=$

Die While Sprache

$$\begin{aligned} S ::= & [x := a]^l \\ & | [\text{skip}]^l \\ & | [S_1; S_2]^l \\ & | \text{if } [b]^l \text{ then } S_1 \text{ else } S_2 \\ & | \text{while } [b]^l \text{ do } S \end{aligned}$$

x Variable Typ `Int` oder `Boolean`

a Ausdruck von passendem Typ zu x

b Ausdruck aus **BExp**

$l \in \mathbf{Lab}$ Label

Stmt : Menge aller syntaktisch korrekten While Programme.

Annahme: Label identifizieren Anweisungen eindeutig!

Beispiel: Syntax

While Program:

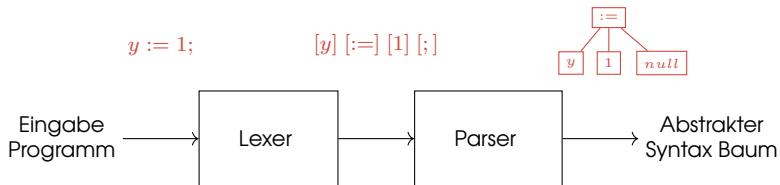
```
[y := x]1;  
[z := 1]2;  
while [y > 1]3 do  
  [z := x * y]4;  
  [y := y - 1]5;  
[y := 0]6
```

Später:

- Def. von Variablen
- Ausführung

Parsen / Übersetzen von While

Compiler Front End:

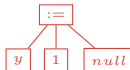


Abstrakter Syntax Baum (AST)

Intuitiv:

Ein abstrakter Syntax Baum ist ein Baum, der die **syntaktische Struktur** eines Programms ausdrückt

- Ausgabe eines Compiler Front-end
- Grundlage für für Generierung von Code



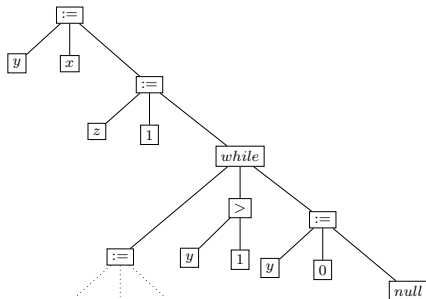
- Dient auch als Grundlage für einfache statische Code-Analyse.
- Z.B.: **Prüfe, dass alle **Namen von Klassen** mit Großbuchstaben beginnen**

AST Beispiel

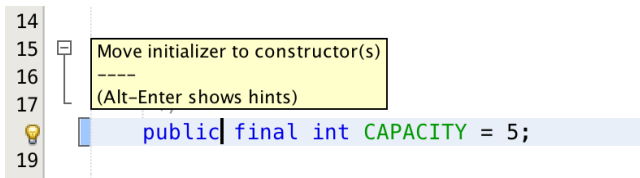
While Programm:

```
[y := x]1;  
[z := 1]2;  
while [y > 1]3 do  
  [z := x * y]4;  
  [y := y - 1]5  
[y := 0]6;
```

Abstrakter Syntax Baum:



AST-basierte Analyse



AST-basierte Analyse

Definition (AST-basierte statische Code-Analyse)

AST-basierte statische Code-Analyse sucht nach Merkmalen / Sttruktur im abstrakten Syntax Baum eines Programms.

Beispiele:

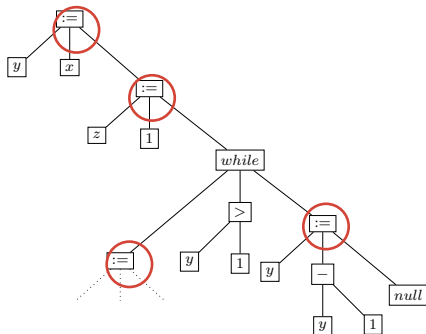
- Initialisierung von Variablen
- Benennung von Klassen
- ...

AST-basierte Analyse: Beispiel

While Programm:

```
[y := x]1;  
[z := 1]2;  
while [y > 1]3 do  
  [z := x * y]4;  
  [y := y - 1]5  
[y := 0]6;
```

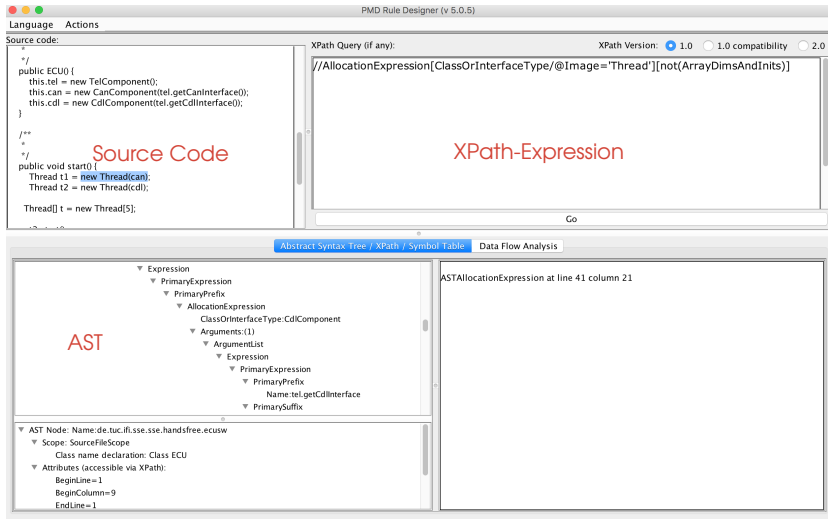
Abstrakter Syntax Baum:



Länge von Variablennamen:

(:=)-Knoten, an denen erstes Kind ein zu kurzes Label (z.B. < 3) hat

PMD Rule Designer



Natürlich funktioniert das auch für Modelle!

Kontrollflussgraph (CFG)

Intuitiv:

Der Kontrollflussgraph eines Programms P ist ein Graph (V, E) mit

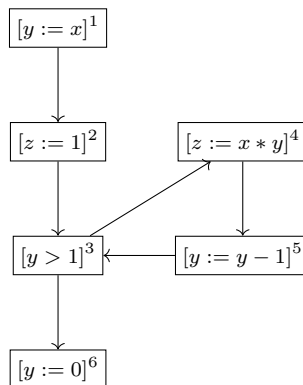
- Menge Knoten V , die einen Knoten für jede Anweisung in P enthält und
- Menge Kanten $E \subseteq V \times V$, die eine Kante $(v, v') \in E$ enthält wenn P von v zu v' übergehen kann.

CFG Beispiel

While Programm:

```
[y := x]1;  
[z := 1]2;  
while [y > 1]3 do  
    [z := x * y]4;  
    [y := y - 1]5  
[y := 0]6;
```

CFG:



Formalisierung Kontrollfluss (1)

Elementare Blöcke eines Programms: $blocks : \mathbf{Stmnt} \rightarrow \mathcal{P}(\mathbf{Blocks})$

$$blocks([x := a]^l) = \{ [x := a]^l \}$$

$$blocks([\mathbf{skip}]^l) = \{ [\mathbf{skip}]^l \}$$

$$blocks([S_1; S_2]^l) = blocks(S_1) \cup blocks(S_2)$$

$$blocks(\mathbf{if} [b]^l \mathbf{then} S_1 \mathbf{else} S_2) = \{ [b]^l \} \cup blocks(S_1) \cup blocks(S_2)$$

$$blocks(\mathbf{while} [b]^l \mathbf{do} S) = \{ [b]^l \} \cup blocks(S)$$

Labels eines Programms: $labels : \mathbf{Stmnt} \rightarrow \mathcal{P}(\mathbf{Lab})$

$$labels(S) = \{ l \in \mathbf{Lab} : [B]^l \in blocks(S) \}$$

Blocks: elementare Blöcke der Form $[x := a]^l$, $[\mathbf{skip}]^l$ bzw. $[b]^l$

Formalisierung Kontrollfluss (2)

Start-Knoten: $init : \text{Stmt} \rightarrow \text{Lab}$

$$init([x := a]^l) = l$$

$$init([\text{skip}]^l) = l$$

$$init([S_1; S_2]^l) = init(S_1)$$

$$init(\text{if } [b]^l \text{ then } S_1 \text{ else } S_2) = l$$

$$init(\text{while } [b]^l \text{ do } S) = l$$

Formalisierung Kontrollfluss (2)

Start-Knoten: $init : \text{Stmt} \rightarrow \text{Lab}$

$$init([x := a]^l) = l$$

$$init([\text{skip}]^l) = l$$

$$init([S_1; S_2]^l) = init(S_1)$$

$$init(\text{if } [b]^l \text{ then } S_1 \text{ else } S_2) = l$$

$$init(\text{while } [b]^l \text{ do } S) = l$$

End-Knoten: $final : \text{Stmt} \rightarrow \mathcal{P}(\text{Lab})$

$$final([x := a]^l) = \{l\}$$

$$final([\text{skip}]^l) = \{l\}$$

$$final([S_1; S_2]^l) = final(S_2)$$

$$final(\text{if } [b]^l \text{ then } S_1 \text{ else } S_2) = final(S_1) \\ \cup final(S_2)$$

$$final(\text{while } [b]^l \text{ do } S) = \{l\}$$

Formalisierung Kontrollfluss (3)

Kontrollfluss: $flow : \mathbf{Stmt} \rightarrow \mathcal{P}(\mathbf{Lab} \times \mathbf{Lab})$

$$flow([x := a]^l) = \emptyset$$

$$flow([\mathbf{skip}]^l) = \emptyset$$

$$flow([S_1; S_2]^l) = \{ (l, init(S_2)) : l \in final(S_1) \}$$

$$flow(\mathbf{if} [b]^l \mathbf{then} S_1 \mathbf{else} S_2) = \{ (l, init(S_1)) \} \\ \cup \{ (l, init(S_2)) \}$$

$$flow(\mathbf{while} [b]^l \mathbf{do} S) = \{ (l, init(S)) \} \\ \cup \{ (l', l) : l' \in final(S) \}$$

Umgekehrter Kontrollfluss: $flow^R : \mathbf{Stmt} \rightarrow \mathcal{P}(\mathbf{Lab} \times \mathbf{Lab})$

$$flow^R(S) = \{ (l, l') : (l', l) \in flow(S) \}$$

Formalisierung Kontrollfluss (4)

Definition (Kontrollflussgraph (CFG))

Der Kontrollflussgraph eines *While* Programms $S_\star$ ist ein Graph (V, E) mit

- Knoten $V = labels(S_\star)$ und
- Kanten $E = flow(S_\star)$.

Annahmen: (vereinfacht Zyklomatische Komplexität)

- Isolierter Start-Knoten: $\forall l \in labels(S_\star) . (l, init(S_\star)) \notin flow(S_\star)$
- Isolierter End-Knoten: $\forall l \in labels(S_\star) . (final(S_\star), l) \notin flow(S_\star)$

(Kann durch Hinzufügen von `skip` Anweisungen erreicht werden)

Zyklomatische Komplexität (CC)

Definition (Zyklomatische Komplexität (CC))

Die Zyklomatische Komplexität eines Programms P entspricht der Anzahl der unabhängigen Pfade in P . Sei e die Zahl der Kanten im Kontrollflussgraph von P , n die Zahl der Knoten im Kontrollflussgraph und p die Zahl der Zusammenhangskomponenten^(*). Dann gilt

$$CC = e - n + 2p$$

- Komplexität wird gemessen durch Anzahl der Pfade
- Viele Pfade $\Leftrightarrow$ hohe Komplexität
- $p = 1$ für Programme mit einem Einstiegspunkt.

(*) : hat z.B. eine Bedeutung für CC eine Java Klasse

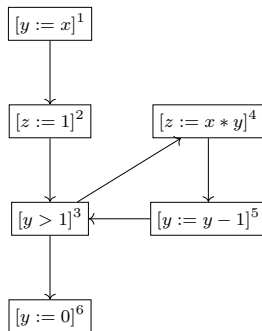
Für SWK irrelevant!

Zyklomatische Komplexität: Beispiel

While Programm:

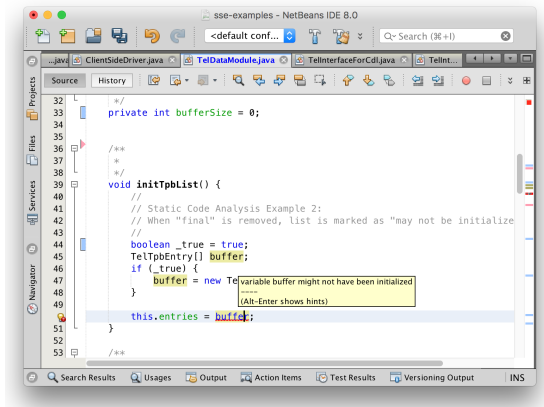
```
[y := x]1;  
[z := 1]2;  
while [y > 1]3 do  
  [z := x * y]4;  
  [y := y - 1]5  
[y := 0]6;
```

Kontrollflussgraph:



$$CC = 6 - 6 + 2 = 2$$

Datenfluss-Analyse



- Manchmal reicht der AST nicht aus ...
- Analyse auf Basis des Kontrollflussgraphen (CFG)

Nicht initialisierte Variable in While Programm

While Programm:

```
[y := 5]1;  
[z := 1]2;  
while [y > 1]3 do  
    [z := x * y]4;  
    [y := y - 1]5  
[y := 0]6;
```

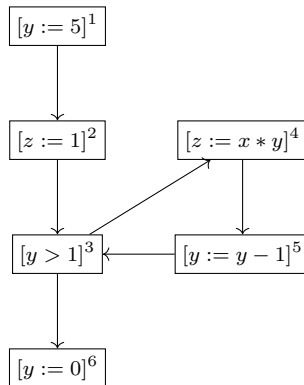
Nicht initialisierte Variable in While Programm

While Programm:

```
[y := 5]1;  
[z := 1]2;  
while [y > 1]3 do  
    [z := x * y]4;  
    [y := y - 1]5  
[y := 0]6;
```

Problem:
x nicht initialisiert!

CFG:



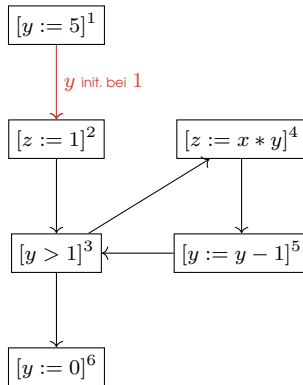
Nicht initialisierte Variable in While Programm

While Programm:

```
[y := 5]1;  
[z := 1]2;  
while [y > 1]3 do  
    [z := x * y]4;  
    [y := y - 1]5  
[y := 0]6;
```

Problem:
x nicht initialisiert!

CFG:



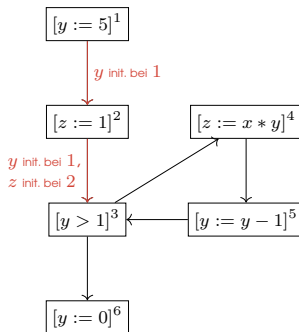
Nicht initialisierte Variable in While Programm

While Programm:

```
[y := 5]1;  
[z := 1]2;  
while [y > 1]3 do  
  [z := x * y]4;  
  [y := y - 1]5  
[y := 0]6;
```

Problem:
x nicht initialisiert!

CFG:



Reaching Definitions

Definition (Reaching Definitions)

Reaching Definitions (RD) berechnet, welche Zuweisungen zu Variablen zu welchen Anweisungen im Programm propagiert werden können.

Anwendung: z.B. Entdecken potenziell uninitialisierter Variablen

Reaching Definitions

Definition (Reaching Definitions)

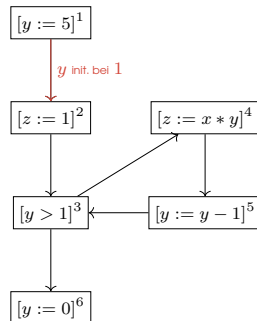
Reaching Definitions (RD) berechnet, welche Zuweisungen zu Variablen zu welchen Anweisungen im Programm propagiert werden können.

Anwendung: z.B. Entdecken potenziell uninitialisierter Variablen

Idee:

Über-approximieren des Datenfluss auf Basis des CFG.

- Über-approximieren: Wir ignorieren Entscheidungen bei Verzweigungen
- Datenfluss: Wir simulieren den Fluss von Information (z.B. Zuweisungen) entlang von Knoten des CFG



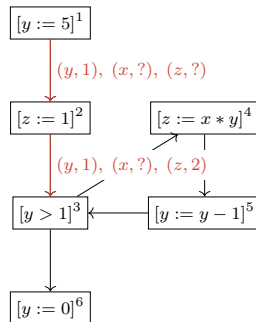
Reaching Definitions: Ansatz

- System von Gleichungen drückt Bedingungen über Datenfluss im CFG aus
- Wir suchen die "kleinste" Lösung, die Bedingungen erfüllt

Definitionen:

Sei (x, l) aus $\mathbf{Var} \times (\mathbf{Lab} \cup \{?\})$.

- Paar (x, l) bedeutet: Anweisung l ändert x
- $(x, ?)$ bedeutet: x ist nicht initialisiert
- Fluss in l : $RD_{entry}(l) \subseteq \mathbf{Var} \times (\mathbf{Lab} \cup \{?\})$
- Fluss aus l : $RD_{exit}(l) \subseteq \mathbf{Var} \times (\mathbf{Lab} \cup \{?\})$



Reaching Definitions: Formalisierung (1)

Kill- und Gen-Funktionen:

$$\begin{aligned} kill_{RD}([x := a]^l) &= \{(x, ?)\} \\ &\cup \{(x, l') : [B]^{l'} \text{ ist Zuweisung zu } x \text{ in } S_\star\} \end{aligned}$$

$$kill_{RD}([\text{skip}]^l) = \emptyset$$

$$kill_{RD}([b]^l) = \emptyset$$

$$gen_{RD}([x := a]^l) = \{(x, l)\}$$

$$gen_{RD}([\text{skip}]^l) = \emptyset$$

$$gen_{RD}([b]^l) = \emptyset$$

Reaching Definitions: Formalisierung (2)

Datenfluss Gleichungen: (Zu verstehen als Bedingungen an Datenfluss)

$$RD_{entry}(l) = \begin{cases} \{ (x, ?) : x \in FV(S_\star) \} & \text{für } l = init(S_\star) \\ \bigcup \{ RD_{exit}(l') : (l', l) \in flow(S_\star) \} & \text{sonst} \end{cases}$$

$$RD_{exit}(l) = (RD_{entry}(l) \setminus kill_{RD}([B]^l)) \cup gen_{RD}([B]^l) \\ \text{für } [B]^l \in blocks(S_\star)$$

Reaching Definitions: Formalisierung (2)

Datenfluss Gleichungen: (Zu verstehen als Bedingungen an Datenfluss)

$$\text{RD}_{\text{entry}}(l) = \begin{cases} \{ (x, ?) : x \in FV(S_\star) \} & \text{für } l = \text{init}(S_\star) \\ \bigcup \{ \text{RD}_{\text{exit}}(l') : (l', l) \in \text{flow}(S_\star) \} & \text{sonst} \end{cases}$$

$$\begin{aligned} \text{RD}_{\text{exit}}(l) &= (\text{RD}_{\text{entry}}(l) \setminus \text{kill}_{\text{RD}}([B]^l)) \cup \text{gen}_{\text{RD}}([B]^l) \\ &\text{für } [B]^l \in \text{blocks}(S_\star) \end{aligned}$$

For $\text{Lab}_\star = \{1, \dots, k\}$

$$\text{RD}(S_\star) = (\text{RD}_{\text{entry}}(1), \text{RD}_{\text{exit}}(1), \dots, \text{RD}_{\text{entry}}(k), \text{RD}_{\text{exit}}(k))$$

$\text{Lab}_\star = \text{labels}(S_\star)$ und
 $FV(S)$: Variablen in Anweisungen und Ausdrücken von S

Reaching Definitions: Einfacher Algorithmus

Gesucht:

Funktionen $reach$ mit $reach_{entry}, reach_{exit} : \mathbf{Lab}_\star \rightarrow \mathcal{P}(\mathbf{Var} \times (\mathbf{Lab} \cup \{?\}))$ die $RD(S_\star)$ erfüllen.

Reaching Definitions: Einfacher Algorithmus

Gesucht:

Funktionen $reach$ mit $reach_{entry}, reach_{exit} : \mathbf{Lab}_\star \rightarrow \mathcal{P}(\mathbf{Var} \times (\mathbf{Lab} \cup \{?\}))$ die $RD(S_\star)$ erfüllen.

Initialisierung:

- $reach_{entry}^0(l) = \{(x, ?) : x \in \mathbf{Var}_\star\}$ für $l = init(S_\star)$
- $reach_{entry}^0(l) = \emptyset$ sonst

Superscripts zählen Schritte des Algorithmus.

Reaching Definitions: Einfacher Algorithmus

Gesucht:

Funktionen $reach$ mit $reach_{entry}, reach_{exit} : \mathbf{Lab}_\star \rightarrow \mathcal{P}(\mathbf{Var} \times (\mathbf{Lab} \cup \{?\}))$ die $RD(S_\star)$ erfüllen.

Initialisierung:

- $reach_{entry}^0(l) = \{(x, ?) : x \in \mathbf{Var}_\star\}$ für $l = init(S_\star)$
- $reach_{entry}^0(l) = \emptyset$ sonst

Vervollständigen:

$$reach_{exit}^i(l) = \begin{cases} reach_{entry}^i(l) \setminus \{(x, l') : l' \in \mathbf{Lab}\} \cup \{(x, l)\} & \text{für } [x := a]^l \\ reach_{entry}^i(l) & \text{sonst} \end{cases}$$

Superscripts zählen Schritte des Algorithmus.

Reaching Definitions: Einfacher Algorithmus

Gesucht:

Funktionen $reach$ mit $reach_{entry}, reach_{exit} : \mathbf{Lab}_\star \rightarrow \mathcal{P}(\mathbf{Var} \times (\mathbf{Lab} \cup \{?\}))$ die $RD(S_\star)$ erfüllen.

Initialisierung:

$$\blacksquare \text{ reach}_{entry}^0(l) = \{(x, ?) : x \in \mathbf{Var}_\star\} \text{ für } l = \text{init}(S_\star)$$

$$\blacksquare \text{ reach}_{entry}^0(l) = \emptyset \text{ sonst}$$

Vervollständigen:

$$\text{reach}_{exit}^i(l) = \begin{cases} \text{reach}_{entry}^i(l) \setminus \{(x, l') : l' \in \mathbf{Lab}\} \cup \{(x, l)\} & \text{für } [x := a]^l \\ \text{reach}_{entry}^i(l) & \text{sonst} \end{cases}$$

Solange instabil:

$$\text{reach}_{entry}^{i+1}(l) = \bigcup_{\{l' \in \mathbf{Lab}_\star : (l', l) \in \text{flow}(S_\star)\}} \text{reach}_{exit}^i(l')$$

Superscripts zählen Schritte des Algorithmus.

Reaching Definitions: Terminierung

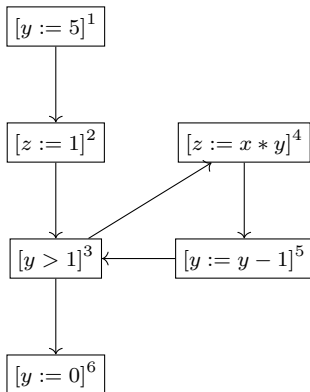
Algorithmus terminiert wenn

$$reach^{i+1} = reach^i$$

Punkt existiert:

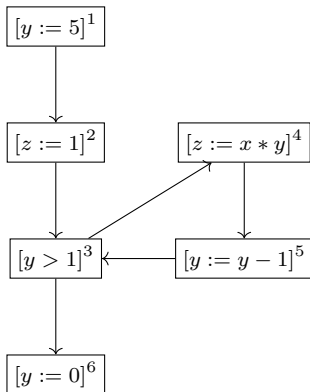
- Nur endlich viele Paare
- Neue Paare (x, l) werden nur einmal hinzugefügt
- Alle Mengen wachsen monoton.

Reaching Definitions: Beispiel



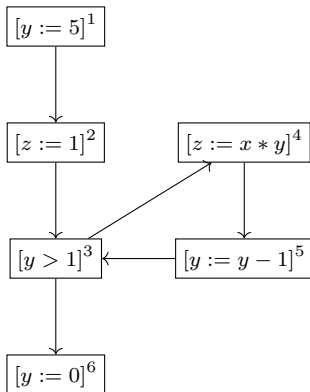
l	$reach_{entry}(l)$	$reach_{exit}(l)$
1	$(x, ?), (y, ?), (z, ?)$	
2		
3		
4		
5		
6		

Reaching Definitions: Beispiel



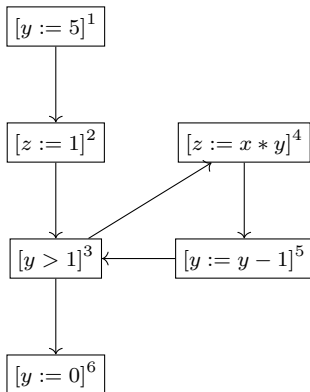
l	$reach_{entry}(l)$	$reach_{exit}(l)$
1	$(x, ?), (y, ?), (z, ?)$	$(x, ?), (y, 1), (z, ?)$
2		$(z, 2)$
3		
4		$(z, 4)$
5		$(y, 5)$
6		$(y, 6)$

Reaching Definitions: Beispiel



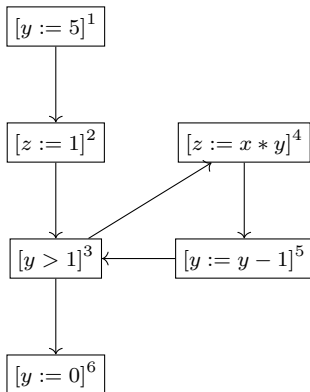
l	$reach_{entry}(l)$	$reach_{exit}(l)$
1	$(x, ?), (y, ?), (z, ?)$	$(x, ?), (y, 1), (z, ?)$
2	$(x, ?), (y, 1), (z, ?)$	$(z, 2)$
3	$(z, 2), (y, 5)$	
4		$(z, 4)$
5	$(z, 4)$	$(y, 5)$
6		$(y, 6)$

Reaching Definitions: Beispiel



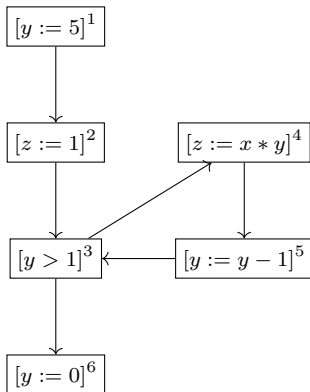
l	$reach_{entry}(l)$	$reach_{exit}(l)$
1	$(x, ?), (y, ?), (z, ?)$	$(x, ?), (y, 1), (z, ?)$
2	$(x, ?), (y, 1), (z, ?)$	$(x, ?), (y, 1), (z, 2)$
3	$(z, 2), (y, 5)$	$(z, 2), (y, 5)$
4		$(z, 4)$
5	$(z, 4)$	$(y, 5), (z, 4)$
6		$(y, 6)$

Reaching Definitions: Beispiel



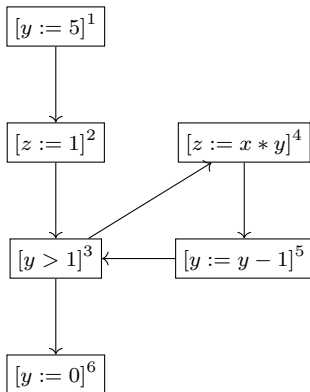
l	$reach_{entry}(l)$	$reach_{exit}(l)$
1	$(x, ?), (y, ?), (z, ?)$	$(x, ?), (y, 1), (z, ?)$
2	$(x, ?), (y, 1), (z, ?)$	$(x, ?), (y, 1), (z, 2)$
3	$(x, ?), (y, 1), (z, 2), (y, 5), (z, 4)$	$(z, 2), (y, 5)$
4	$(z, 2), (y, 5)$	$(z, 4)$
5	$(z, 4)$	$(y, 5), (z, 4)$
6	$(z, 2), (y, 5),$	$(y, 6)$

Reaching Definitions: Beispiel



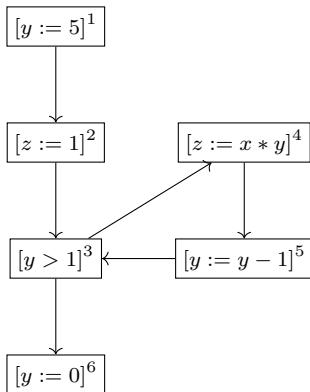
l	$reach_{entry}(l)$	$reach_{exit}(l)$
1	$(x, ?), (y, ?), (z, ?)$	$(x, ?), (y, 1), (z, ?)$
2	$(x, ?), (y, 1), (z, ?)$	$(x, ?), (y, 1), (z, 2)$
3	$(x, ?), (y, 1), (z, 2), (y, 5), (z, 4)$	$(x, ?), (y, 1), (z, 2), (y, 5), (z, 4)$
4	$(z, 2), (y, 5)$	$(z, 4), (y, 5)$
5	$(z, 4), (y, 5)$	$(y, 5), (z, 4)$
6	$(z, 2), (y, 5),$	$(y, 6), (z, 2)$

Reaching Definitions: Beispiel



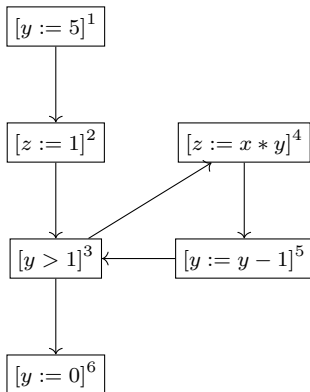
l	$reach_{entry}(l)$	$reach_{exit}(l)$
1	$(x, ?), (y, ?), (z, ?)$	$(x, ?), (y, 1), (z, ?)$
2	$(x, ?), (y, 1), (z, ?)$	$(x, ?), (y, 1), (z, 2)$
3	$(x, ?), (y, 1), (z, 2), (y, 5), (z, 4)$	$(x, ?), (y, 1), (z, 2), (y, 5), (z, 4)$
4	$(x, ?), (y, 1), (z, 2), (y, 5), (z, 4)$	$(z, 4), (y, 5)$
5	$(z, 4), (y, 5)$	$(y, 5), (z, 4)$
6	$(x, ?), (y, 1), (z, 2), (y, 5), (z, 4)$	$(y, 6), (z, 2)$

Reaching Definitions: Beispiel



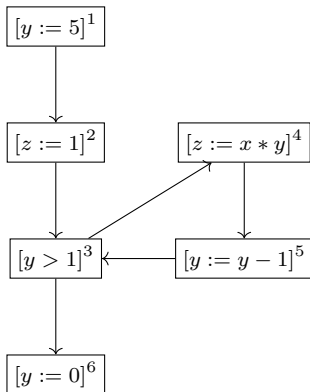
l	$reach_{entry}(l)$	$reach_{exit}(l)$
1	$(x, ?), (y, ?), (z, ?)$	$(x, ?), (y, 1), (z, ?)$
2	$(x, ?), (y, 1), (z, ?)$	$(x, ?), (y, 1), (z, 2)$
3	$(x, ?), (y, 1), (z, 2), (y, 5), (z, 4)$	$(x, ?), (y, 1), (z, 2), (y, 5), (z, 4)$
4	$(x, ?), (y, 1), (z, 2), (y, 5), (z, 4)$	$(x, ?), (y, 1), (z, 4), (y, 5)$
5	$(z, 4), (y, 5)$	$(y, 5), (z, 4)$
6	$(x, ?), (y, 1), (z, 2), (y, 5), (z, 4)$	$(x, ?), (y, 6), (z, 2), (z, 4)$

Reaching Definitions: Beispiel



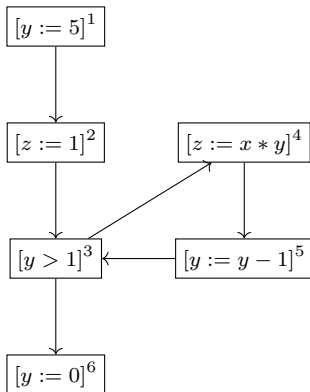
l	$reach_{entry}(l)$	$reach_{exit}(l)$
1	$(x, ?), (y, ?), (z, ?)$	$(x, ?), (y, 1), (z, ?)$
2	$(x, ?), (y, 1), (z, ?)$	$(x, ?), (y, 1), (z, 2)$
3	$(x, ?), (y, 1), (z, 2), (y, 5), (z, 4)$	$(x, ?), (y, 1), (z, 2), (y, 5), (z, 4)$
4	$(x, ?), (y, 1), (z, 2), (y, 5), (z, 4)$	$(x, ?), (y, 1), (z, 4), (y, 5)$
5	$(x, ?), (y, 1), (z, 4), (y, 5)$	$(y, 5), (z, 4)$
6	$(x, ?), (y, 1), (z, 2), (y, 5), (z, 4)$	$(x, ?), (y, 6), (z, 2), (z, 4)$

Reaching Definitions: Beispiel



l	$reach_{entry}(l)$	$reach_{exit}(l)$
1	$(x, ?), (y, ?), (z, ?)$	$(x, ?), (y, 1), (z, ?)$
2	$(x, ?), (y, 1), (z, ?)$	$(x, ?), (y, 1), (z, 2)$
3	$(x, ?), (y, 1), (z, 2), (y, 5), (z, 4)$	$(x, ?), (y, 1), (z, 2), (y, 5), (z, 4)$
4	$(x, ?), (y, 1), (z, 2), (y, 5), (z, 4)$	$(x, ?), (y, 1), (z, 4), (y, 5)$
5	$(x, ?), (y, 1), (z, 4), (y, 5)$	$(x, ?), (y, 5), (z, 4)$
6	$(x, ?), (y, 1), (z, 2), (y, 5), (z, 4)$	$(x, ?), (y, 6), (z, 2), (z, 4)$

Reaching Definitions: Beispiel

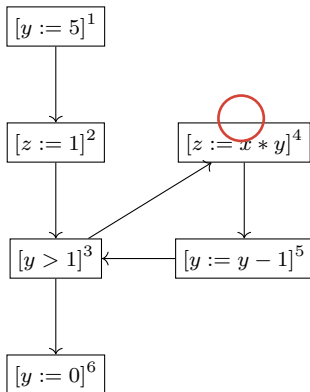


l	$reach_{entry}(l)$	$reach_{exit}(l)$
1	$(x, ?), (y, ?), (z, ?)$	$(x, ?), (y, 1), (z, ?)$
2	$(x, ?), (y, 1), (z, ?)$	$(x, ?), (y, 1), (z, 2)$
3	$(x, ?), (y, 1), (z, 2), (y, 5), (z, 4)$	$(x, ?), (y, 1), (z, 2), (y, 5), (z, 4)$
4	$(x, ?), (y, 1), (z, 2), (y, 5), (z, 4)$	$(x, ?), (y, 1), (z, 4), (y, 5)$
5	$(x, ?), (y, 1), (z, 4), (y, 5)$	$(x, ?), (y, 5), (z, 4)$
6	$(x, ?), (y, 1), (z, 2), (y, 5), (z, 4)$	$(x, ?), (y, 6), (z, 2), (z, 4)$

$$RD_{entry}(l) = \left\{ \bigcup \{ (x, ?) : x \in FV(S_*) \} \text{ für } l = init(S_*) \right. \\ \left. \bigcup \{ RD_{exit}(l') : (l', l) \in flow(S_*) \} \text{ sonst} \right.$$

$$RD_{exit}(l) = (RD_{entry}(l) \setminus kill_{RD}([B]^l)) \cup gen_{RD}([B]^l)$$

Reaching Definitions: Beispiel



l	$reach_{entry}(l)$	$reach_{exit}(l)$
1	$(x, ?), (y, ?), (z, ?)$	$(x, ?), (y, 1), (z, ?)$
2	$(x, ?), (y, 1), (z, ?)$	$(x, ?), (y, 1), (z, 2)$
3	$(x, ?), (y, 1), (z, 2), (y, 5), (z, 4)$	$(x, ?), (y, 1), (z, 2), (y, 5), (z, 4)$
4	$(x, ?), (y, 1), (z, 2), (y, 5), (z, 4)$	$(x, ?), (y, 1), (z, 4), (y, 5)$
5	$(x, ?), (y, 1), (z, 4), (y, 5)$	$(x, ?), (y, 5), (z, 4)$
6	$(x, ?), (y, 1), (z, 2), (y, 5), (z, 4)$	$(x, ?), (y, 6), (z, 2), (z, 4)$

$$RD_{entry}(l) = \left\{ \bigcup \left\{ (x, ?) : x \in FV(S_{\star}) \right\} \text{ für } l = init(S_{\star}) \right. \\ \left. \bigcup \left\{ RD_{exit}(l') : (l', l) \in flow(S_{\star}) \right\} \right\} \text{sonst}$$

$$RD_{exit}(l) = (RD_{entry}(l) \setminus kill_{RD}([B]^l)) \cup gen_{RD}([B]^l)$$

Potential use of
uninitialized variable!

Reaching Definitions: Korrektheit

Zu zeigen:

Wenn in einer **Ausführung** von $S_\star$, die Zuweisung zu x von Anweisung l bis zu Anweisung l' überlebt, dann ist (x, l) in $reach_{entry}(l')$ von jeder gültigen Lösung $reach$ für $RD(S_\star)$.

Weitere Analysen

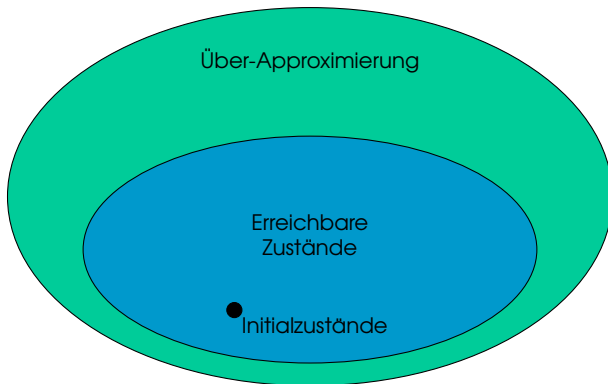
- Available Expressions

Welche Ausdrücke werden wann berechnet und werden an welchen anderen Stellen im Programm noch benötigt

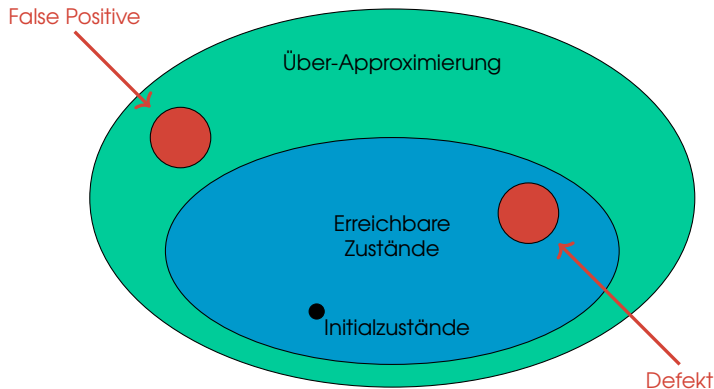
- Live Variables

Welche Variablen werden zu welchem Zeitpunkt noch benötigt

'False Positives' bei statischer Analyse mit Über-Approximierung



'False Positives' bei statischer Analyse mit Über-Approximierung



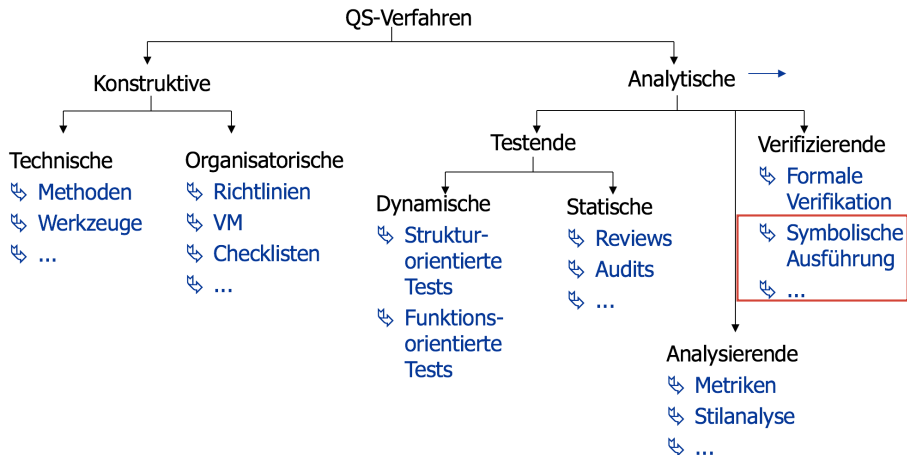
Datenfluss-Analyse: Falsch Positive Beispiele

Tafel

Agenda

1. Motivation und Begriffe
2. (Code) Standards
3. (Code) Reviews
4. Modellierung und Generierung
5. Verifikation: Model-Checking und Invarianten
6. Metriken und Statische Code Analyse
- 7. Formale Semantik and Symbolische Ausführung**
8. Literatur und Standards

QS - Klassifizierung von Ansätzen



Semantik einer Programmiersprache

- In frühen Programmiersprachen war die Semantik in natürlicher Sprache definiert
- Konsequenz: Ungenauigkeit, nicht definiertes Verhalten

Beispiel:

```
integer procedure awkward
begin
  x := x+1
  awkward := 3
end awkward
...
x := 4
z := x+awkward
```

- Globale Variable x.
- Was ist der Wert von $x + \text{awkward}$?

Formale Semantik

Idee: **Formale** Semantik für Programme, die Bedeutung (= was bei der Ausführung passiert) eindeutig festlegt.

Strukturierte Operationelle Semantik:

- Strukturiert: Feste Regeln in strukturierter Darstellung
- Operationell: Beschreibt die Ausführung
- Semantik: Bedeutung der Syntax eines Programms

Beispiel:

- While Sprache.

Formale Semantik für While

Wir benötigen:

- "Abstrakte Maschine": Formaler Begriff einer Zustandes
- Konfiguration: Kombination aus Programm und Zustand
- Ausführung: Übergänge zwischen Kombinationen

Zustand = Belegung von Variablen

Definition (Belegung (Wdh.))

Eine Belegung v von X ist eine Typ-konsistente Zuweisung von Werten zu Variablen in X . v ist eine Funktion und für $x \in X$ ist $v(x)$ der Wert mit dem x in v belegt ist.

Sei V_X die Menge aller möglichen typ-konsistenten Belegungen von X .

Update des Zustandes:

Wir schreiben: $v' = v[x \mapsto e]$ für:

$$v'(y) = \begin{cases} v(y) & \text{wenn } y \neq x, \\ e & \text{wenn } y = x. \end{cases}$$

Wobei: $x \in X$ und e konkreter Wert, der zum Typ von x passt.

Konfigurationen

Definition (Konkrete Konfiguration)

Eine **konkrete Konfiguration** ist ein Tupel $\langle S, v \rangle$ aus

- einem Programmtext $S \in \mathbf{Stmt}$ und
- einer Belegung $v \in V_X$.

Beispiel:

- Konfiguration $\sigma = \langle \text{if } (x > 0) \text{ then } x := 0 \text{ else skip, } \{x \mapsto 2\} \rangle$

Idee:

- S speichert den noch auszuführenden Teil des Programms
- v speichert den aktuellen Wert aller Variablen

Wir nehmen im folgenden immer an, dass $X = \text{Vars}(S_*)$
wenn wir das While Programm S_* analysieren.

Auswertung von Ausdrücken

- Set von Variablen X
- Ausdrücke aus $\mathbf{AExp} \cup \mathbf{BExp}$ über Variablen aus X

Auswertung:

- Funktion $\alpha_X : \mathbf{AExp} \times V_X \mapsto \mathbb{Z}$
- Funktion $\beta_X : \mathbf{BExp} \times V_X \mapsto \{0, 1\}$

Beispiel (Negation):

- $\beta_X[(\neg b), v] = 1$ gdw. $\beta_X[b, v] = 0$ (sonst 0)

Übergangsregeln

Wir definieren einige Regeln für das Abarbeiten von Anweisungen

Inferenzregeln:

Format: $\frac{\text{Prämissen}}{\text{Konklusion}}$

Beispiel: $\frac{a \Rightarrow b, b \Rightarrow c}{a \Rightarrow c}$

Axiome:

Format: $\overline{\text{Axiom}}$ (oder einfach: Axiom)

Beispiel: $n + 0 = n$

- Generell haben Prämisse, Konklusion und Axiome die Form $A \Rightarrow B$.
- Bedeutung: A kann in B überführt werden.
- Dabei sind A und B Konfigurationen oder Zustände

SOS: Skip

Das Axiom SKIP gibt an, wie eine `skip` Anweisung ausgeführt wird:

$$\langle \text{skip}, v \rangle \Rightarrow v \quad [\text{SKIP}]$$

- Die Konfiguration $\langle \text{skip}, v \rangle$ wird in den Zustand v überführt.
- Die Regel überführt eine Konfiguration in einen Zustand
- Nach der Überführung verbleibt nichts auszuführen

Beispiel:

$$\langle \text{skip}, \{x \mapsto 2\} \rangle \Rightarrow \{x \mapsto 2\} \quad [\text{SKIP}]$$

SOS: Zuweisung

Das Axiom ASS gibt an, wie eine Zuweisung $x := a$ ausgeführt wird:

$$\langle x := a, v \rangle \Rightarrow v[x \mapsto c] \quad \text{für } c = \alpha_X[a, v] \quad [\text{ASS}]$$

- Die Konfiguration $\langle x := a, v \rangle$ wird in den Zustand $v[x \mapsto c]$ überführt.
- Die Regel überführt eine Konfiguration in einen Zustand
- Nach der Überführung verbleibt nichts auszuführen

Beispiel:

$$\langle x := 5 + x, \{x \mapsto 2\} \rangle \Rightarrow \{x \mapsto 7\} \quad [\text{ASS}]$$

SOS: Kontrollstrukturen

Die Axiome IF_T und IF_F geben an, wie eine `if then else` Anweisung ausgeführt wird:

$$\langle \text{if } b \text{ then } S_1 \text{ else } S_2, v \rangle \Rightarrow \langle S_1, v \rangle \quad \text{falls } \beta_X[b, v] = 1 \quad [\text{IF}_T]$$

$$\langle \text{if } b \text{ then } S_1 \text{ else } S_2, v \rangle \Rightarrow \langle S_2, v \rangle \quad \text{falls } \beta_X[b, v] = 0 \quad [\text{IF}_F]$$

- Die Konfiguration $\langle \text{if } b \text{ then } S_1 \text{ else } S_2, v \rangle$ wird in die Konfiguration $\langle S_1, v \rangle$ überführt falls in s die Bedingung b gilt; falls b nicht gilt wird nach $\langle S_2, v \rangle$ überführt
- Die Regeln überführen eine Konfiguration in eine Konfiguration
- Nach der Überführung verbleibt etwas auszuführen

Beispiel:

$$\langle \text{if } (x > 0) \text{ then } x := 0 \text{ else skip}, \{x \mapsto 2\} \rangle \Rightarrow \langle x := 0, \{x \mapsto 2\} \rangle \quad [\text{IF}_T]$$

SOS: Schleifen

Die Axiome WH_T und WH_F geben an, wie eine Schleife ausgeführt wird. Die Schleife wird durch die Regeln einmal "abgerollt":

$$\langle \text{while } b \text{ do } S, v \rangle \Rightarrow \langle S; \text{while } b \text{ do } S, v \rangle \quad \text{falls } \beta_X[b, v] = 1 \quad [\text{WH}_T]$$

$$\langle \text{while } b \text{ do } S, v \rangle \Rightarrow v \quad \text{falls } \beta_X[b, v] = 0 \quad [\text{WH}_F]$$

- Die Regel WH_T überführt eine Konfiguration in eine Konfiguration. Nach der Überführung verbleibt etwas auszuführen.
- Die Regel WH_F überführt eine Konfiguration in einen Zustand. Nach der Überführung verbleibt nichts auszuführen.

Beispiel:

$$\begin{aligned} &\langle \text{while } (x > 0) \text{ do } x := x - 1, \{x \mapsto 2\} \rangle \Rightarrow \\ &\langle x := x - 1; \text{while } (x > 0) \text{ do } x := x - 1, \{x \mapsto 2\} \rangle \quad [\text{WH}_T] \end{aligned}$$

SOS: Komposition

Inferenzregeln für Komposition geben an, wie Sequenzen von Anweisungen abzuarbeiten sind. Diese Regeln werden immer mit einem Axiom kombiniert:

$$\frac{\langle S_1, v \rangle \Rightarrow \langle S'_1, v' \rangle}{\langle S_1; S_2, v \rangle \Rightarrow \langle S'_1; S_2, v' \rangle} \quad [\text{COMP}_1]$$

$$\frac{\langle S_1, v \rangle \Rightarrow v'}{\langle S_1; S_2, v \rangle \Rightarrow \langle S_2, v' \rangle} \quad [\text{COMP}_2]$$

- Regel COMP_1 funktioniert mit Regeln IF_T , IF_F und WH_T
- Regel COMP_2 funktioniert mit ASS und SKIP und WH_F
- Nach der Überführung verbleibt etwas auszuführen

Beispiel:

$$\frac{\langle x := 1, \{x \mapsto 2\} \rangle \Rightarrow \{x \mapsto 1\}}{\langle x := 1; \text{skip}, \{x \mapsto 2\} \rangle \Rightarrow \langle \text{skip}, \{x \mapsto 1\} \rangle} \quad [\text{COMP}_2 + \text{ASS}]$$

Beispiel

While Program:

```
[y := x]1;  
[z := 1]2;  
while [y > 1]3 do  
  [z := x * y]4;  
  [y := y - 1]5;  
[y := 0]6
```

Abkürzungen:

- S_i : Anweisung mit Label i

Zustand:

- $v_1 = \{x \mapsto 2, y \mapsto 0, z \mapsto 0\}$
- $v_2 = \{x \mapsto 2, y \mapsto 2, z \mapsto 0\}$
- $v_3 = \{x \mapsto 2, y \mapsto 2, z \mapsto 1\}$
- $v_4 = \{x \mapsto 2, y \mapsto 1, z \mapsto 4\}$

Beispiel

While Program:

```
[y := x]1;  
[z := 1]2;  
while [y > 1]3 do  
  [z := x * y]4;  
  [y := y - 1]5;  
[y := 0]6
```

[COMP₂ + ASS] :

$$\frac{\langle y := x, v_1 \rangle \Rightarrow v_1[y \mapsto 2]}{\langle y := x; S_2; S_3; S_6, v_1 \rangle \Rightarrow \langle S_2; S_3; S_6, v_1[y \mapsto 2] \rangle}$$

Abkürzungen:

- S_i : Anweisung mit Label i

Zustand:

- $v_1 = \{x \mapsto 2, y \mapsto 0, z \mapsto 0\}$
- $v_2 = \{x \mapsto 2, y \mapsto 2, z \mapsto 0\}$
- $v_3 = \{x \mapsto 2, y \mapsto 2, z \mapsto 1\}$
- $v_4 = \{x \mapsto 2, y \mapsto 1, z \mapsto 4\}$

Weitere Beispiele auf Papier ...

Beispiel

While Program:

```
[y := x]1;  
[z := 1]2;  
while [y > 1]3 do  
  [z := x * y]4;  
  [y := y - 1]5;  
[y := 0]6
```

[COMP₂ + ASS] :

$$\frac{\langle y := x, v_1 \rangle \Rightarrow v_1[y \mapsto 2]}{\langle y := x; S_2; S_3; S_6, v_1 \rangle \Rightarrow \langle S_2; S_3; S_6, v_1[y \mapsto 2] \rangle}$$

[COMP₂ + ASS] :

$$\frac{\langle z := 1, v_2 \rangle \Rightarrow v_2[z \mapsto 1]}{\langle z := 1; S_3; S_6, v_2 \rangle \Rightarrow \langle S_3; S_6, v_2[z \mapsto 1] \rangle}$$

Abkürzungen:

- S_i : Anweisung mit Label i

Zustand:

- $v_1 = \{x \mapsto 2, y \mapsto 0, z \mapsto 0\}$
- $v_2 = \{x \mapsto 2, y \mapsto 2, z \mapsto 0\}$
- $v_3 = \{x \mapsto 2, y \mapsto 2, z \mapsto 1\}$
- $v_4 = \{x \mapsto 2, y \mapsto 1, z \mapsto 4\}$

Weitere Beispiele auf Papier ...

Beispiel

While Program:

```
[y := x]1;  
[z := 1]2;  
while [y > 1]3 do  
  [z := x * y]4;  
  [y := y - 1]5;  
[y := 0]6
```

Abkürzungen:

- S_i : Anweisung mit Label i

Zustand:

- $v_1 = \{x \mapsto 2, y \mapsto 0, z \mapsto 0\}$
- $v_2 = \{x \mapsto 2, y \mapsto 2, z \mapsto 0\}$
- $v_3 = \{x \mapsto 2, y \mapsto 2, z \mapsto 1\}$
- $v_4 = \{x \mapsto 2, y \mapsto 1, z \mapsto 4\}$

[COMP₂ + ASS] :

$$\frac{\langle y := x, v_1 \rangle \Rightarrow v_1[y \mapsto 2]}{\langle y := x; S_2; S_3; S_6, v_1 \rangle \Rightarrow \langle S_2; S_3; S_6, v_1[y \mapsto 2] \rangle}$$

[COMP₂ + ASS] :

$$\frac{\langle z := 1, v_2 \rangle \Rightarrow v_2[z \mapsto 1]}{\langle z := 1; S_3; S_6, v_2 \rangle \Rightarrow \langle S_3; S_6, v_2[z \mapsto 1] \rangle}$$

[COMP₁ + WH_T] ($y > 1$) :

$$\frac{\langle S_3, v_3 \rangle \Rightarrow \langle S_4; S_5; S_3, v_3 \rangle}{\langle S_3; S_6, v_3 \rangle \Rightarrow \langle S_4; S_5; S_3; S_6, v_3 \rangle}$$

Beispiel

While Program:

```
[y := x]1;  
[z := 1]2;  
while [y > 1]3 do  
  [z := x * y]4;  
  [y := y - 1]5;  
[y := 0]6
```

Abkürzungen:

- S_i : Anweisung mit Label i

Zustand:

- $v_1 = \{x \mapsto 2, y \mapsto 0, z \mapsto 0\}$
- $v_2 = \{x \mapsto 2, y \mapsto 2, z \mapsto 0\}$
- $v_3 = \{x \mapsto 2, y \mapsto 2, z \mapsto 1\}$
- $v_4 = \{x \mapsto 2, y \mapsto 1, z \mapsto 4\}$

[COMP₂ + ASS] :

$$\frac{\langle y := x, v_1 \rangle \Rightarrow v_1[y \mapsto 2]}{\langle y := x; S_2; S_3; S_6, v_1 \rangle \Rightarrow \langle S_2; S_3; S_6, v_1[y \mapsto 2] \rangle}$$

[COMP₂ + ASS] :

$$\frac{\langle z := 1, v_2 \rangle \Rightarrow v_2[z \mapsto 1]}{\langle z := 1; S_3; S_6, v_2 \rangle \Rightarrow \langle S_3; S_6, v_2[z \mapsto 1] \rangle}$$

[COMP₁ + WH_T] ($y > 1$) :

$$\frac{\langle S_3, v_3 \rangle \Rightarrow \langle S_4; S_5; S_3, v_3 \rangle}{\langle S_3; S_6, v_3 \rangle \Rightarrow \langle S_4; S_5; S_3; S_6, v_3 \rangle}$$

...

[COMP₂ + WH_F] ($y \leq 1$) :

$$\frac{\langle S_3, v_4 \rangle \Rightarrow v_4}{\langle S_3; S_6, v_4 \rangle \Rightarrow \langle S_6, v_4 \rangle}$$

Beispiel

While Program:

```
[y := x]1;  
[z := 1]2;  
while [y > 1]3 do  
  [z := x * y]4;  
  [y := y - 1]5;  
[y := 0]6
```

Abkürzungen:

- S_i : Anweisung mit Label i

Zustand:

- $v_1 = \{x \mapsto 2, y \mapsto 0, z \mapsto 0\}$
- $v_2 = \{x \mapsto 2, y \mapsto 2, z \mapsto 0\}$
- $v_3 = \{x \mapsto 2, y \mapsto 2, z \mapsto 1\}$
- $v_4 = \{x \mapsto 2, y \mapsto 1, z \mapsto 4\}$

[COMP₂ + ASS] :

$$\frac{\langle y := x, v_1 \rangle \Rightarrow v_1[y \mapsto 2]}{\langle y := x; S_2; S_3; S_6, v_1 \rangle \Rightarrow \langle S_2; S_3; S_6, v_1[y \mapsto 2] \rangle}$$

[COMP₂ + ASS] :

$$\frac{\langle z := 1, v_2 \rangle \Rightarrow v_2[z \mapsto 1]}{\langle z := 1; S_3; S_6, v_2 \rangle \Rightarrow \langle S_3; S_6, v_2[z \mapsto 1] \rangle}$$

[COMP₁ + WH_T] ($y > 1$) :

$$\frac{\langle S_3, v_3 \rangle \Rightarrow \langle S_4; S_5; S_3, v_3 \rangle}{\langle S_3; S_6, v_3 \rangle \Rightarrow \langle S_4; S_5; S_3; S_6, v_3 \rangle}$$

...

[COMP₂ + WH_F] ($y \leq 1$) :

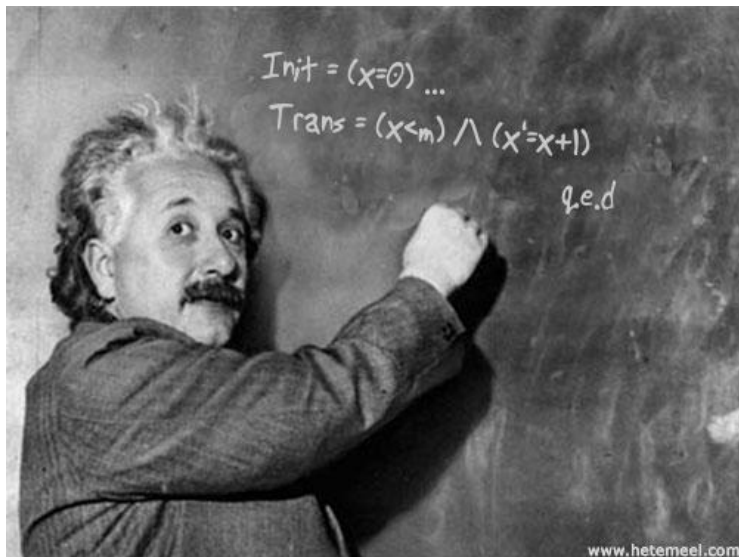
$$\frac{\langle S_3, v_4 \rangle \Rightarrow v_4}{\langle S_3; S_6, v_4 \rangle \Rightarrow \langle S_6, v_4 \rangle}$$

[ASS] :

$$\langle y := 0, v_4 \rangle \Rightarrow v_4[y \mapsto 0]$$

Weitere Beispiele auf Papier ...

Symbolische Analyse



Symbolische Ausführung

Definition (Symbolische Ausführung)

Symbolische Ausführung ist eine statische Analyse, bei der ein Programm mit symbolischen Werten anstelle von konkreten Werten ausgeführt wird.

- Alle Anweisungen werden symbolisch berechnet.
- Bei der Ausführung von Kontrollstrukturen können ggf. beide Zweige ausgeführt werden.
- Die auf einem Pfad der Ausführung entstehenden Einschränkungen von Variablen werden als Pfadbedingungen aufgezeichnet.

Beispiel:

Concrete: $i = 1$

```
int y = i + 100; // y = 101
if (y > 200) {
    ...
} else {
    ... // here
}
```

Symbolic: $i = X$

```
int y = i + 100; // y = X+100
if (y > 200) {
    ... // X+100 > 200
} else {
    ... // X+100 <= 200
}
```

SE: Konfigurationen

Symbolische Belegung:

- Menge symbolischer Variablen $\mathbf{Sym} = \{A, B, \dots\}$
- Symbolische Belegung $\mathbf{v} : X \mapsto \mathbb{F}_{\mathbf{Sym}}$
- $\mathfrak{V}_X$: Menge aller symbolischen Belegungen von X

Symbolisches Auswerten:

Für $X = \{x_1, \dots, x_k\}$ und $a \in \mathbb{F}_X$ ist $\sigma_X : \mathbb{F}_X \times \mathfrak{V} \mapsto \mathbb{F}_{\mathbf{Sym}}$ so dass:

$$\sigma_X[a, \mathbf{v}] =_{def} a[\mathbf{v}(x_1)/x_1][\dots][\mathbf{v}(x_k)/x_k] \in \mathbb{F}_{\mathbf{Sym}}$$

Beispiel:

- Sei $\mathbf{v} = \{x_1 \mapsto (A + 2)\}$ und $a = 'x_1 > 0'$
- Dann ist $\sigma_X[a, \mathbf{v}] = '(A + 2) > 0'$

SE: Skip

Das Axiom SKIP^{Sym} gibt an, wie eine `skip` Anweisung symbolisch ausgeführt wird:

$$\langle \text{skip}, v, \varphi \rangle \Rightarrow (v, \varphi) \quad [\text{SKIP}^{\text{Sym}}]$$

- Die Konfiguration $\langle \text{skip}, v, \varphi \rangle$ wird in den symbolischen Zustand (v, φ) überführt.
- Die Regel überführt eine Konfiguration in einen Zustand
- Nach der Überführung verbleibt nichts auszuführen

Beispiel:

$$\langle \text{skip}, \{x \mapsto (A + 2)\}, (A > 10) \rangle \Rightarrow (\{x \mapsto (A + 2)\}, (A > 10)) \quad [\text{SKIP}^{\text{Sym}}]$$

SE: Zuweisung

Das Axiom ASS^{Sym} gibt an, wie eine Zuweisung $x := a$ symbolisch ausgeführt wird:

$$\langle x := a, \mathbf{v}, \varphi \rangle \Rightarrow (\mathbf{v}[x \mapsto t], \varphi) \quad \text{für } t = \sigma_X[a, \mathbf{v}] \quad [\text{ASS}^{\text{Sym}}]$$

- Die Konfiguration $\langle x := a, \mathbf{v}, \varphi \rangle$ wird in den Zustand $(\mathbf{v}[x \mapsto t], \varphi)$ überführt.
- Die Regel überführt eine Konfiguration in einen Zustand
- Nach der Überführung verbleibt nichts auszuführen

Beispiel:

$$\langle x := 5 + x, \{x \mapsto (A + 2)\}, \text{true} \rangle \Rightarrow (\{x \mapsto 5 + (A + 2)\}, \text{true}) \quad [\text{ASS}^{\text{Sym}}]$$

SE: Kontrollstrukturen

Die Axiome IF_T^{Sym} und IF_F^{Sym} geben an, wie eine `if then else` Anweisung symbolisch ausgeführt wird:

$$\langle \text{if } b \text{ then } S_1 \text{ else } S_2, v, \varphi \rangle \Rightarrow \langle S_1, v, (\varphi \wedge \sigma_X[b, v]) \rangle \quad \text{falls } \varphi \wedge \sigma_X[b, v] \text{ sat.} \quad [\text{IF}_T^{\text{Sym}}]$$

$$\langle \text{if } b \text{ then } S_1 \text{ else } S_2, v, \varphi \rangle \Rightarrow \langle S_2, v, (\varphi \wedge \sigma_X[\neg b, v]) \rangle \quad \text{falls } \varphi \wedge \sigma_X[\neg b, v] \text{ sat.} \quad [\text{IF}_F^{\text{Sym}}]$$

- Beachte: Es können oft beide Regeln angewendet werden.
Es entsteht ein symbolischer Ausführungsbaum!
- Die Regeln überführen eine symbolische Konfiguration in eine symbolische Konfiguration. Nach der Überführung verbleibt etwas auszuführen

Beispiel:

$$\langle \text{if } (x > 0) \text{ then } x := 0 \text{ else skip, } \{x \mapsto A\}, \text{true} \rangle \Rightarrow \langle x := 0, \{x \mapsto A\}, (A > 0) \rangle \quad [\text{IF}_T^{\text{Sym}}]$$

$$\langle \text{if } (x > 0) \text{ then } x := 0 \text{ else skip, } \{x \mapsto A\}, \text{true} \rangle \Rightarrow \langle \text{skip, } \{x \mapsto A\}, \neg(A > 0) \rangle \quad [\text{IF}_F^{\text{Sym}}]$$

Symbolische Pfadbedingung $\varphi \in \mathbb{F}_{\text{Sym}}$

SE: Schleifen

Die Axiome WH_T^{Sym} und WH_F^{Sym} geben an, wie eine Schleife symbolisch ausgeführt wird. Die Schleife wird durch die Regeln einmal "abgerollt":

$$\langle \text{while } b \text{ do } S, v, \varphi \rangle \Rightarrow \langle S; \text{while } b \text{ do } S, v, (\varphi \wedge \sigma_X[b, v]) \rangle \quad \text{falls } \varphi \wedge \sigma_X[b, v] \text{ sat.} \quad [WH_T^{Sym}]$$

$$\langle \text{while } b \text{ do } S, v, \varphi \rangle \Rightarrow (v, (\varphi \wedge \sigma_X[\neg b, v])) \quad \text{falls } \varphi \wedge \sigma_X[\neg b, v] \text{ sat.} \quad [WH_F^{Sym}]$$

- Die Regel WH_T^{Sym} überführt eine symbolische Konfiguration in eine symbolische Konfiguration. Nach der Überführung verbleibt etwas auszuführen.
- Die Regel WH_F^{Sym} überführt eine symbolische Konfiguration in einen symbolischen Zustand. Nach der Überführung verbleibt nichts auszuführen.

Beispiel:

$$\begin{aligned} &\langle \text{while } (x > 0) \text{ do } x := x - 1, \{x \mapsto A\}, (A > 1) \rangle \Rightarrow \\ &\langle x := x - 1; \text{while } (x > 0) \text{ do } x := x - 1, \{x \mapsto A\}, (A > 1 \wedge A > 0) \rangle \quad [WH_T^{Sym}] \end{aligned}$$

Regel WH_F^{Sym} im Beispiel nicht anwendbar, da $((A > 1) \wedge \neg(A > 0))$ nicht erfüllbar!

SE: Komposition

Inferenzregeln für Komposition geben an, wie Sequenzen von Anweisungen abzuarbeiten sind. Diese Regeln werden immer mit einem Axiom kombiniert:

$$\frac{\langle S_1, \mathbf{v}, \varphi \rangle \Rightarrow \langle S'_1, \mathbf{v}', \varphi' \rangle}{\langle S_1; S_2, \mathbf{v}, \varphi \rangle \Rightarrow \langle S'_1; S_2, \mathbf{v}', \varphi' \rangle} \quad [\text{COMP}_1^{\text{Sym}}]$$

$$\frac{\langle S_1, \mathbf{v}, \varphi \rangle \Rightarrow (\mathbf{v}', \varphi')}{\langle S_1; S_2, \mathbf{v}, \varphi \rangle \Rightarrow \langle S_2, \mathbf{v}', \varphi' \rangle} \quad [\text{COMP}_2^{\text{Sym}}]$$

- Regel $\text{COMP}_1^{\text{Sym}}$ funktioniert mit Regeln IF_T^{Sym} , IF_F^{Sym} und WH_T^{Sym}
- Regel $\text{COMP}_2^{\text{Sym}}$ funktioniert mit ASS^{Sym} und SKIP^{Sym} und WH_F^{Sym}

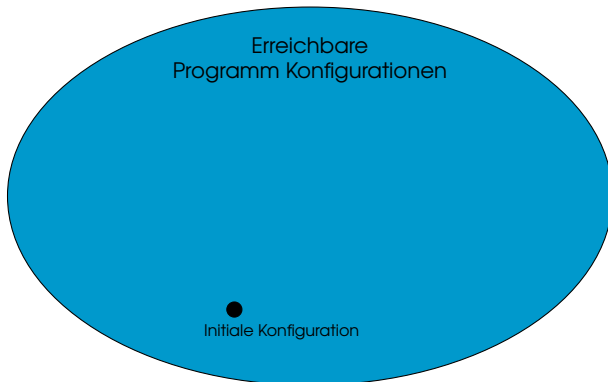
Beispiel:

$$\frac{\langle x := 1, \{x \mapsto A\}, \text{true} \rangle \Rightarrow (\{x \mapsto 1\}, \text{true})}{\langle x := 1; \text{skip}, \{x \mapsto A\}, \text{true} \rangle \Rightarrow \langle \text{skip}, \{x \mapsto 1\}, \text{true} \rangle} \quad [\text{COMP}_2^{\text{Sym}} + \text{ASS}^{\text{Sym}}]$$

Beispiel

Auf Papier

Symbolische Ausführung: Intuition



Symbolische Ausführung: Intuition



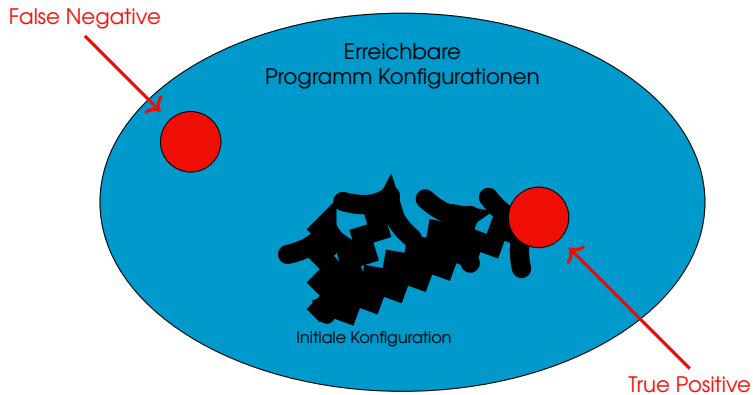
Symbolische Ausführung: Intuition



Symbolische Ausführung: Intuition



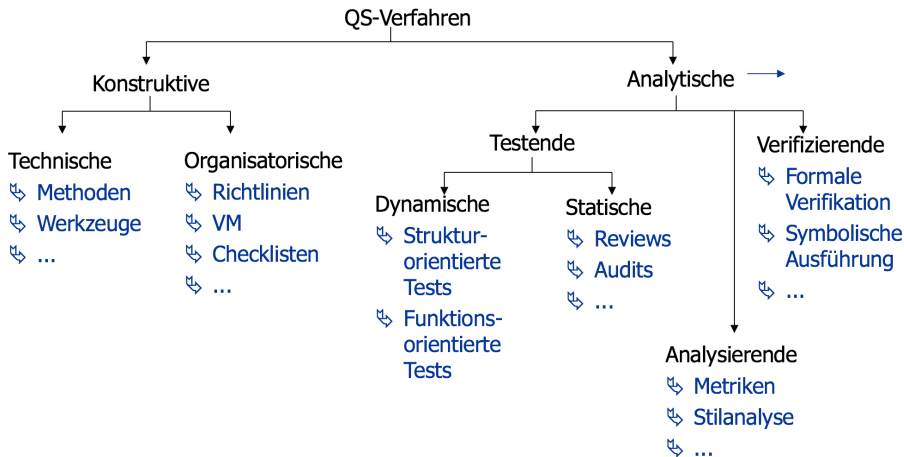
Symbolische Ausführung: Intuition



Agenda

1. Motivation und Begriffe
2. (Code) Standards
3. (Code) Reviews
4. Modellierung und Generierung
5. Verifikation: Model-Checking und Invarianten
6. Metriken und Statische Code Analyse
7. Formale Semantik and Symbolische Ausführung
8. Literatur und Standards

QS - Klassifizierung von Ansätzen



■ Testen → SWT

- Ranjit Jhala and Rupak Majumdar: Software Model Checking ACM Computing Surveys (CSUR) Volume 41 Issue 4, October 2009.
- Rajeev Alur: Principles of Cyber-Physical Systems. Mit Press, 2015.
- Thomas Kühne: Matters of (Meta-) Modeling, STTT (2006) 5: 369. Springer, 2006.
- Clark Barrett, Roberto Sebastiani, Sanjit A. Seshia and Cesare Tinelli: Satisfiability Modulo Theories. In: Armin Biere, Marijn Heule, Hans van Maaren and Toby Walsch. Handbook of satisfiability. IOS Press, 2008.

- Dirk W. Hoffmann. Software-Qualität, 2008.
<https://link.springer.com/book/10.1007%2F978-3-540-76323-9>
- Liggesmeyer: Software-Qualität Testen, Analysieren und Verifizieren von Software; Springer Spektrum, 2009.
- Nielson, Nielson, Hankin: Principles of Program Analysis; Springer 1999.
- King: Symbolic Execution and Program Testing; Communications of the ACM (CACM), 19 (7), 1976.

- **ISO/IEC 25010:2011** Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models
- **ISO/IEC 25000** System und Software-Engineering - Qualitätskriterien und Bewertung von System- und Softwareprodukten (SQuaRE) - Leitfaden für SQuaRE
- **ISO 9126** Software engineering — Product quality was an international standard for the evaluation of software quality (Ersetzt durch ISO/IEC 25000)
- **DIN EN ISO 9000:2015-11** Qualitätsmanagementsysteme – Grundlagen und Begriffe

- **ISO/IEC 19508:2014** Information technology - Meta Object Facility (MOF)
- **OMG MDA** Model Driven Architecture
- **OMG 2007** OMG: Unified Modeling Language: Superstructure, Version 2.1.1. OMG document formal/2007-02-05.